

SetUp OpenEphys

How to launch Open-ephys :

- if you want to look at the former data already recorded (perceval or others..) :
 - go to x2goclient session on hubel (where is all the data we need (perceval)
 - do : `/home/programs/open-ephys/open-ephys`
 - it will open the app like that (note that is it important the numbers of plugin on the right, some are vital for the process of the data) :



- add a file reader and we are looking for the dat in the doc :
hubel/data-131/perceval/**Raw/**
 - then click until you find in the file you want to analyze a structure.oebin
- if you want to record and have the detection live :
 - open-ephys on your computer, be sure to have the latest version

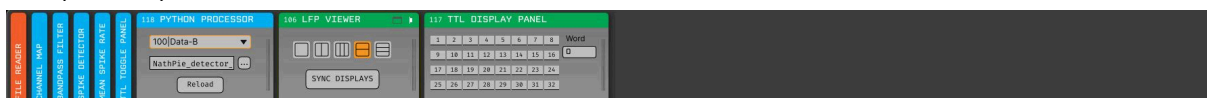
How to get the right signal chain

The signal chain is where you can put your plugins (things on the left of the screen that you can slide on the bottom of the screen).

Please follow the instructions and use the same parameters. You can see here the result :

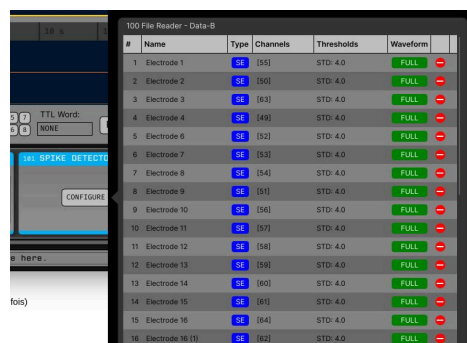


and (la suite) :



1. There is 2 choice for the first plugin :
 - a. **“File Reader”** : if you want to read data already recorded (reminder : if so you have to be on the open-ephys of hubel to get the data from hubel). You selected your file (**taking the Raw then the structure.oebin of the session** (list of possible session : **sleepm tachment sleep tachment sleeps**) corresponding to an hour of three day of recording)

- b. OTHERWISE you will use the plugin **“Acquisition Board”** to get the data
2. **“Channel map”** : **ordonner les channels** :
- Get the .xml done from the excel of Antoine that precises which channel corresponds to which one in “reality” and gives the packages of channels that are in the NR, the PFC and the Hippocampus
 - remark : to have the right order, you should do recording of x minutes then study it to do the excel that Antine is doing to have the translation between which channel number is where. And this before doing the real detection (pre-experience!)
 - and then you use this conversion to write the channel mapping in the .json here is an example of how the plugin channel mapping is reading it : “channel_mapping_Rat004_karadoc” at
hubel/media/data-103/Nathan/Avalanche_project/aval_data/
 - use one program python ? to get the file .json of the channel mapping
 - be aware of which group begins in the list (what are the 32 first channels ? by default should be the ones of the NR)
 - be aware of which channel is not used
 - be aware of which channel correspond to the accelerometers components
 - you can now import this file in the channel map plugin
 - remark : open-ephys starts the counting at 1 BUT in .py or other doc it starts at 0 (so there is a +1 on the open-ephys interface at each number or position of channels)**
 - number = “the name of the channel”**
 - position = “the position regarding to other channels”**
3. if you are in case 1.b : **“Record Node”** to get the data for post analysis (Antoine knows how to do it)
- IMPORTANT REMARK** : where to put the Record Node ? Before or after the channel mapping ? Do you want your raw datas to be in order in advance
4. **“Bandpass Filter”** : filter the channels that you are interested in. ATTENTION : **do not filter the channels corresponding to the accelerometer component !!** 300Hz to 6000Hz.
5. **“Spike Detector”** : Here are the parameters STD 4, take all the channels from the NR

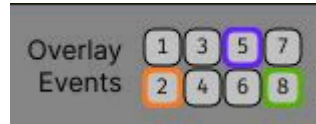


#	Name	Type	Channels	Thresholds	Waveform
1	Electrode 1	SE	[55]	STD: 4.0	FULL
2	Electrode 2	SE	[50]	STD: 4.0	FULL
3	Electrode 3	SE	[63]	STD: 4.0	FULL
4	Electrode 4	SE	[49]	STD: 4.0	FULL
5	Electrode 5	SE	[52]	STD: 4.0	FULL
6	Electrode 7	SE	[53]	STD: 4.0	FULL
7	Electrode 8	SE	[54]	STD: 4.0	FULL
8	Electrode 9	SE	[51]	STD: 4.0	FULL
9	Electrode 10	SE	[56]	STD: 4.0	FULL
10	Electrode 11	SE	[57]	STD: 4.0	FULL
11	Electrode 12	SE	[58]	STD: 4.0	FULL
12	Electrode 13	SE	[59]	STD: 4.0	FULL
13	Electrode 14	SE	[60]	STD: 4.0	FULL
14	Electrode 15	SE	[61]	STD: 4.0	FULL
15	Electrode 16	SE	[64]	STD: 4.0	FULL
16	Electrode 16 (1)	SE	[62]	STD: 4.0	FULL

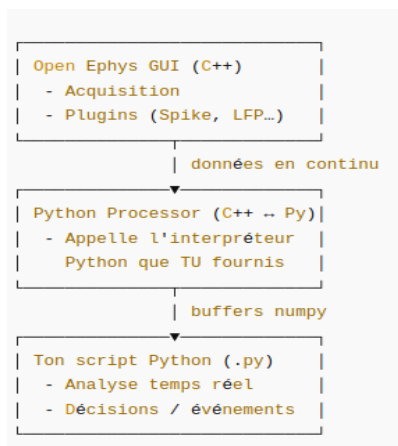
6. **“Mean Spike Rate”** : Select all the channels that you have analyzed in the Spike Detector (hence from the Nucleus Reunien).

- a. Time constant : 100ms. Output is where (which channel) the mean spike rate will be written.
 - b. ATTENTION : this is overwriting on other data, be sure that the output channel is not one already used.
 - i. maybe adding a new channel is possible ?
 - ii. BUT if we put mean spike rate **after** record node, does the output of meanspike rate (channel that its position is written in the plugin mean spike rate in "Output") overwrite the channel where it is? For instance i am putting my mean spike rate on the channel at the position 33 (that we suppose was carrying other data before doing that), is this channel overwritten by those new values and we will recording those new values instead of the formers ones ? Or as the record node is **before** the mean spike rate in the signal chain there is no issue with that and we will keep recording the original values of this channel ?
7. "TTL TOGGLE PANEL" : if it is not here : CTRL+P and install it and its sister TTL DISPLAY PANEL they should appear together in your plugin list.
- a. this is your interface as an user of the detector of InfraSlow Rhythm. When you click on a number it created a TTL event. While you have not turned it off (by clicking again), the event is still considered as "True" (it is important).
 - b. You can click on **nb 10** to start the integration of the thresholds, and it will after 20 sec of integration start to detect avalanches and Infralow.
 - i. **This is a key point in the detection : the moment where you decide to start the detecting program. Those first 20 seconds after your click are vital because determine the calibration of the detection.**
 - ii. **Thus when you should click** : when those conditions are all fulfilled at the same time :
 1. the rat is asleep and is not moving
 2. When you look at the mean Spike rate and it seems that it is currently in an infralowrhythm (succession of ON/OFF)
 - iii. After having clicked on, **be sure to click off the button in less than 20 seconds**. Other wise the program will restart on and on the integration after the 20 seconds.
 - iv. **If it seems that the integration went badly and thus the detection is bad too : click again on 10 and it will restart the integration and the detection.**
 - c. You can click on **nb 15** to stop Infralow (put the event Infralow from True to False) and demand to rebegin the detection to this point.
8. "Python Processor" : again it is not here CTRL+P etc.. Select the program : **NathPie_detector_user_mode.py** from
 hubel/media/data-103/Nathan/Avalanche_project/aval_code/ (you can copy it locally)
- a. be sure to read the python Processor Section
 - b. Normally you should get your Python Environment by selecting the folder "miniconda3" at
 Users>Acquisition\miniconda3\envs\oe-python-plugin

9. **“LFP Viewer”** : use the split screen to have on the top the visual of the channels and on the bottom the visual of the only channel corresponding to mean spike rate (ie the output channel of this plugin)
10. **“TTL Display Panel”** : to know which event is True or False at the very moment.
 - a. Something good to know : You can use on the bottom screen the Overlay Events to see filters that show if currently we are in avalanche (green nb 8), InfraSlow (violet nb5) and Wake/REM (orange nb 2)



How to have the Python Processor :



- # Initialiser conda pour bash (une seule fois)
`~/miniconda3/bin/conda init bash`
`source ~/.bashrc`
`conda --version`
`conda activate oe-python-plugin`

- # Créer un environnement Python 3.10 pour Open Ephys
`conda create -n oe-python-plugin python=3.10`

- # Activer l'environnement

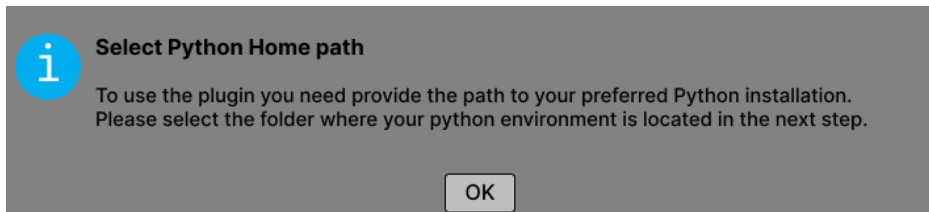
`conda activate oe-python-plugin`
 (si besoin de quitter l'env : `conda deactivate`)

- # Vérifier la version
`python --version`

- # Installer numpy
`conda install numpy`

En gros : <https://open-ephys.github.io/gui-docs/User-Manual/Plugins/Python-Processor.html>

- avant de lancer quoi que ce soit il faut lancer l'env virtuel (l'activer) pour que conda soit actif
- on met le python processor dans la chain signal : il demande quel environnement il peut utiliser. on choisit le dossier de l'environnement open-ephys ou il ya le bon python !



- Puis on sélectionne au chemin qu'on veut le programme python .py (plugin)

trouver où j'ai mis python de cette version

- soit dans hubel/home/programs/ il ya python
- soit dans hubel/home/nathan/miniconda3

How to modify/get access to the Python Processor code:

Pour la mise en place :

- voir dernière partie (juste au dessus)

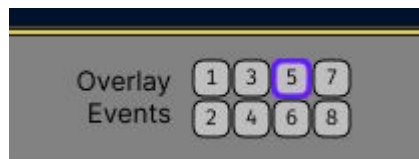
Pour des modifications :

- On ouvre son code python dans vscode (ex : code /media/data-103/Nathan/Avalanche_project/aval_code/avalanche_detector.py)
- **pour chaque modification apportée au code on doit faire CTRL C + /home/programs/open-ephys/open-ephys (relancer open-ephys)**
- ou faire /home/nathan/open-ephys/open-ephys_new/open-ephys

Sinon : <https://open-ephys.github.io/gui-docs/User-Manual/Plugins/Python-Processor.html>

Manip good to know :

- les print sont très hasardeux ne pas essayer de s'y fier (ou alors envoyer des demande de print conséquentes sinon pas visibles)
 - il vaut mieux utiliser **self.processor.add_python_event(4, True)** avec une condition du type if: créé un événement que tu peux voir en mettant visible en cliquant sur (5 ici par ex, car décalage de 1 avec les output)

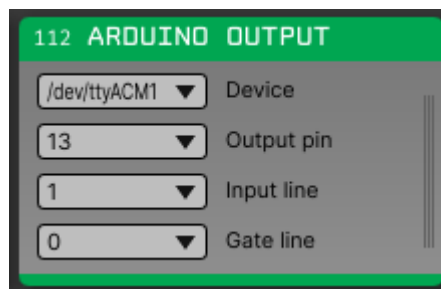


- une session qui beug : press enter et ca la remet bien (ca fait on/off acquisition) !
 - sinon tu ouvres dans le terminal un autre onglet puis tu fermes l'ancien (MAIS PERTE DES DONNEES de l'ancien)

Setup Opto System

How to connect O-E and Arduino

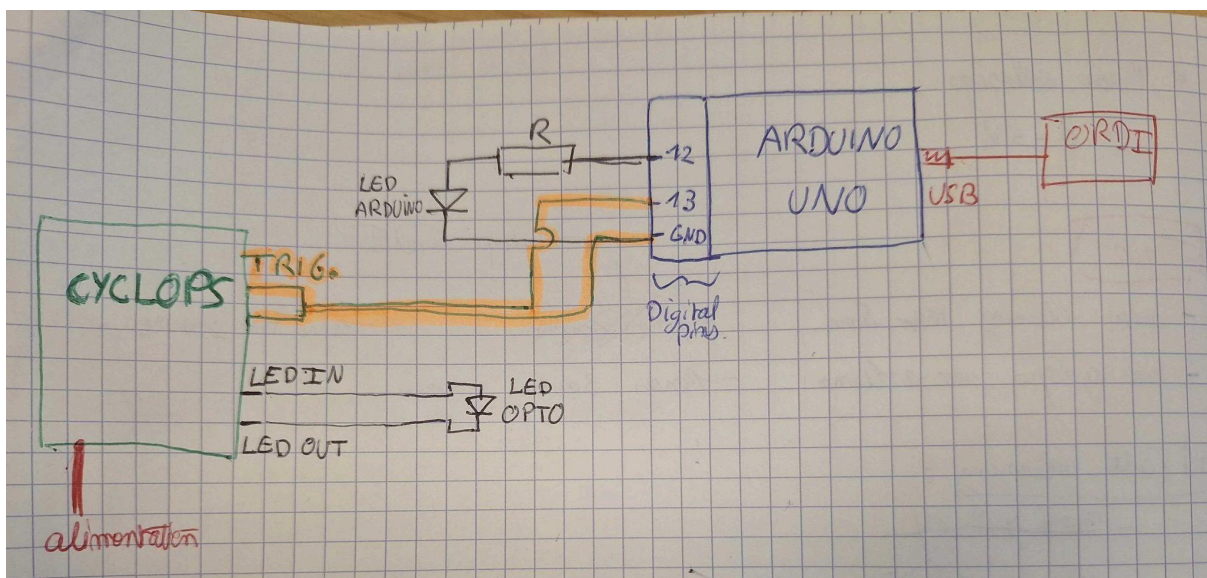
- charge your open-ephys locally (on your computer)
 - add the signal chain corresponding (cf SetUp Open-ephys)
 - /\ open-ephys locally won't work if you try to read files because they are on hubel and mounting large amount of data is too much and graphically very bad.
 - read files of already recorded session on hubel :
/home/programs/open-ephys/open-ephys
- add to the end of the signal chain :
 - the ARDUINO OUTPUT plugin :



- Output pin : c'est le signal envoyé au pin numéro 13 ici
- Input line : numéro de l'événement TTL (nous ça serait l'événement correspondant à oui/non InfraSlow state)
- Device : brancher USB connecté à Arduino à l'ordi, normalement le plugin le détecte et tu le sélectionnes dans device
- Programme :
 - le programme utilisé se trouve dans :
hubel/media/data-103/Nathan/Avalanche_project/aval_code/Arduino&Cyclops /
 - se nomme "**StandardFirmata_with_Indicator.ino**"
 - permet d'activer en 0-5 V le pin de Output pin
 - **et** d'activer le pin numéro 12 pour allumer la LED lorsque pin 13 est HIGH (donc quand on est dans une période de stimulation (IS_state))
 - à retrouver dans les lignes 412 du programme la fonction stimulationlogic
 - ligne 45 pour changer les pins qui reçoivent l'instruction (jusqu'ici le 13) et pour envoyer stimulation (jusqu'ici le 12)
 - il est normalement déjà compilé dans la carte Arduino UNO donc pas besoin de le recompiler (s'il le faut c'est sur ARduino IDE, j'ai dit des choses à ce sujet sur Advances)
- Branchement électroniques : voir schéma des branchements à la fin de ces pages

How to connect Arduino to Cyclops:

- connecter le pin 13 (et le ground) au TRIG. de Cyclops
 - envoie le 0-5V à une carte Teensy (un peu comme Arduino), le message de oui ou non en analogique (0V = Non au stimuli pour nous et 5V = Oui au stimuli)
- On peut compiler sur la Teensy le programme situé à `hubel/media/data-103/Nathan/Avalanche_project/aval_code/Arduino&Cyclops/teensy_cyclops_wave/` qui se nomme justement **teensy_cyclops_wave.ino**
 - **attention** : il faut que les programmes Cyclops.cpp et Cyclops.h soient bien situés avec le code .ino sinon le code pourrait aller chercher des fonctions dans les deux autres.
 - DONC s'il faut copier pour coller dans votre local, ALORS vous copier le dossier entier teensy_cyclops_wave/
 - **Possibilité de modification de l'amplitude (AMPLITUDE) et de la fréquence de stimulus sinusoïdal (FREQ) dans les premières lignes**
 - **ne pas compiler lorsque l'état event TTL sur Open-ephys est déclaré comme ON sinon ça inverse tout** (risque s'active au off et s'éteint au on)
 - mon programme démarre un cosinus (donc sinusoïdale) au moment où le trigger est déclenché (5V)
- brancher la LED à LED OUT
 - **regarder les spécificités (nombre de LEDs et ce qu'elles admettent en courant max) pour pas la/les cramer**
 - théoriquement pas besoin de résistance car les LED que l'on utilise (OSRAM PLT3 450GB) devraient être ok pour ça



REMARQUE : if you want to measure with oscilloscope : put it on REF OUT of the cyclops to see the result of the voltage you are creating.

SetUp Codes

How to interpret the DATA : Matlab :

- 2 codes exist on Matlab with 2 functionalities at the location :
hubel/media/data-103/Nathan/Avalanche_project/aval_code/
- first code : **Sessions_analyse_UsAvals_Nath.m**
 - to get the DATA from an animal, one day and analyse on session (of a phases like for instance "sleepm")
 - to look at mean firing rate/ IS period / avalanches for this session
 - determine the appropriate start time to use in the Open Ephys File Reader
- second code : **Comparison_OE_Matlab_detection.m**
 - to get the DATA from an animal, one day and analyse on session (of a phases like for instance "sleepm")
 - to get the timestamps of your detection from the open-ephys programme
NathPie_detector_user_mode
 - Temporally align Open Ephys detections with IS and avalanche intervals obtained from Pietro's MATLAB algorithm
 - Sometimes you need **to manually remove the last interval** in **IS_OE_intervals** and **wakeREM_OE_intervals**. In some cases, the Open Ephys output contains only the start timestamp (first column) without a corresponding end timestamp. This results in an invalid interval where the computed end time equals the **start_sec_decay** offset and becomes smaller than the start time, which is not physically possible. Such incomplete intervals must be discarded before running the comparison analysis.
 - Compute statistical metrics to quantitatively compare Open Ephys detection with MATLAB (Pietro) detection

NathPie_detector : usage

LOCALISATION :

- Where to find it : **hubel/media/data-103/Nathan/Avalanche_project/aval_code/**
- Its name : **NathPie_detector_user_mode.py**
 - a. copy it on your local machine (your computer), not not modify the official version
- If it is not clear in english : **NathPie_detector_user_mode_inFrench.py**
(commentaires originaux)

TENET :

Il permet de détecter et suivre les **Infraslow Avalanches (IS)** à partir de la fréquence de spikes et des données d'accéléromètre. Il envoie également des événements TTL pour

indiquer l'état des avalanches, le rythme InfraSlow et l'état Wake/REM. L'utilisateur peut interagir via le panneau TTL pour recalculer les seuils ou arrêter immédiatement la détection IS.

SETUP :

1. cf Setup Open-ephys for the signal chain to setup in open-ephys gui
2. open the python programme (on vscode for instance)
3. check the parameters at the section "modifiable parameters"
 - a. see what is next for more precision
 - b. modification depending of the rat / channels (electrode and accelerometer) / computer
4. **not forget to save your changes on your computer**
5. then on open-ephys : fetch your program with the plugin python processor
 - a. if there is a problem the open-ephys will tell you
6. run the acquisition with the recording

HOW IT WORKS :

Processor operates through several successive stages:

1. First, **data acquisition** and moving average computation are performed during 20 seconds (self.time_limit_integration, it is a parameter free to be changed if necessary). Spike rate values and acceleration signals are extracted from the selected channels. A moving average is then computed over the last seconds defined by **time_value_moy** for spike rate and **time_value_acceleration** for acceleration. These smoothed signals are used for all subsequent detections in order to reduce high-frequency fluctuations and improve stability.
2. **User control** is implemented via the TTL toggle panel (cf open-ephys plugin). One TTL line, **ligne_threshold_integration** (default value 9 on python , corresponding to the **10** on the TTL PANEL), is dedicated to triggering threshold recalculation. When the user activates this line, a new threshold integration phase is initiated and all parameters of the last integration disappear to make place for new thresholds. Another TTL line, **ligne_stop_IS_now** (default value 14 on python , corresponding to the **15** on the TTL PANEL), allows the user to immediately stop IS detection. When this line is activated, all ongoing detections and related states are reset.
 - a. VITAL PART : where the user should press the button 10 :
 - i. **Those first 20 seconds after your click are vital because determine the calibration of the detection**
 - ii. **Thus when you should click** : when those conditions are all fulfilled at the same time :

1. the rat is asleep and is not moving
 2. When you look at the mean Spike rate and it seems that it is currently in an infraslowrhythm (succession of ON/OFF)
 - iii. After having clicked on, be sure to click off the button in less than 20 seconds. Other wise the program will restart on and on the integration after the 20 seconds.
 - iv. If it seems that the integration went badly and thus the detection is bad too : click again on 10 and it will restart the integration and the detection.
3. **Threshold computation** occurs after accumulating values during `time_limit_integration` seconds. The spike rate threshold (used after to detect whether or not we are in ON or OFF avalanche) is defined as the percentile specified by `percentage`. The acceleration threshold is defined as the percentile specified by `percentage_accel` (again to define on the accelerometer data whether or not the rat is awake or else different than nREM). In addition, a baseline acceleration level at rest is computed and used later for Wake/REM state detection.
4. **Two things :**
- a. **Wake/REM detection** is triggered when the moving average acceleration exceeds the acceleration threshold or when the spike rate becomes excessively high. In this case, the system switches to a Wake/REM state. Wake/REM periods are recorded, and any ongoing IS or ON detection is reset during these periods to prevent false IS classification. If we are not in the state we can enter in the Avalanche detection and InfraSlow detection
 - b. **Avalanche detection** is based on transitions between OFF and ON states. The transition from OFF to ON is triggered when the moving average spike rate exceeds the spike rate threshold. At this moment, the beginning of the avalanche is recorded in `start_time_ON`, and the previous OFF interval is closed by storing `end_time_OFF`. The transition from ON to OFF occurs when the moving average spike rate falls below the threshold. The end of the avalanche is recorded in `end_time_ON`. If the avalanche duration exceeds `max_time_ON`, it is considered invalid and IS detection is reset. If the duration is within acceptable limits, the corresponding OFF interval duration is computed and stored in `duration_OFF`. But if an ON period is too small we consider it as noise detection and we still are in OFF period
5. **InfraSlow (IS) detection** is based on the temporal structure of OFF and ON intervals. An IS period is detected when at least two OFF intervals (thus 2 ON) are longer than `min_time_OFF`, and none of the OFF intervals exceeds `max_time_OFF` (the selection of max and min ON period is already done in 4.). When these conditions are satisfied, the IS state is updated and the corresponding TTL events are sent. Wake/REM state flags are also updated accordingly.

THE PARAMETERS :

1. The parameter `time_limit_integration` defines the duration in seconds during which spike rate and acceleration values are accumulated to compute detection thresholds. For example, a value of 20.0 means that thresholds are estimated from the first 20 seconds of data (or from the moment threshold recalculation is triggered). A longer integration time produces a more statistically stable threshold but delays detection readiness. A shorter integration time makes the system more reactive but potentially less robust to noise.
2. The parameter `percentage` defines the percentile used to compute the spike rate threshold. For example, a value of 40 means that the spike rate threshold corresponds to the 40th percentile of the spike rate distribution during the integration window. Lower values make the system more permissive, increasing ON detections, whereas higher values make detection more selective.
 - a. For now i have tested this parameter to be optimized for best detection (cf “Reglage du modèle”) at 40
3. The parameter `percentage_accel` defines the percentile used to compute the acceleration threshold. For example, a value of 70 means that the acceleration threshold corresponds to the 70th percentile of the acceleration distribution during integration. Increasing this value makes Wake/REM detection more conservative, while decreasing it increases sensitivity to movement.
 - a. For now i have tested this parameter to be optimized for best detection (cf “Reglage du modèle”) at 70
4. The parameter `max_time_ON` defines the maximum allowed duration, in seconds, for an avalanche (ON state). For example, 14 seconds means that any ON period longer than 14 seconds will be considered physiologically invalid and ignored. When this limit is exceeded, IS detection is reset. This parameter prevents pathological or sustained high-activity periods from being misclassified as valid avalanches.
 - a. For now i have tested this parameter to be optimized for best detection (cf “Reglage du modèle”)
5. The parameter `min_time_ON` defines the minimum duration required for an avalanche to be considered valid. For example, 0.3 seconds means that very brief threshold crossings shorter than 300 ms will be ignored. This prevents noise-driven fluctuations from being classified as true ON events.
 - a. For now i have tested this parameter to be optimized for best detection (cf “Reglage du modèle”)
6. The parameter `max_time_OFF` defines the maximum allowed duration for an OFF interval. For example, 16 seconds means that if an OFF period exceeds 16 seconds, it breaks the temporal structure required for InfraSlow detection. This parameter ensures that IS periods are composed of relatively regular OFF intervals and not separated by excessively long silent phases.
 - a. For now i have tested this parameter to be optimized for best detection (cf “Reglage du modèle”)
7. The parameter `min_time_OFF` defines the minimum duration required for an OFF interval to be counted toward IS detection. For example, 0.5 seconds means that very short OFF gaps are ignored. This avoids counting micro-interruptions in activity as structurally meaningful OFF periods.

- a. For now i have tested this parameter to be optimized for best detection (cf "Reglage du modèle")
8. The parameter `time_value_moy` defines the time window, in seconds, used to compute the moving average of the spike rate. For example, 8 seconds means that each decision of whether or not there is a too high intensity of spike is based on the mean spike rate over the last 8 seconds. A larger window increases stability but reduces temporal precision. A smaller window increases responsiveness but may introduce variability.
 - a. For now i have tested this parameter to be optimized for best detection (cf "Reglage du modèle")
9. The parameter `time_value_acceleration` defines the time window used to compute the moving average of acceleration. For example, 8 seconds means that movement detection is based on smoothed acceleration over 8 seconds. As with spike rate smoothing, larger values increase stability while smaller values increase sensitivity to rapid changes.
 - a. For now i have tested this parameter to be optimized for best detection (cf "Reglage du modèle")
10. The parameter `multiplier_tresh` is a multiplicative factor applied to the spike rate threshold. For example, a value of 3 means that the computed percentile threshold is multiplied by 3 before being used for detection. Increasing this value makes ON detection more stringent; decreasing it increases sensitivity.
 - a. For now i have tested this parameter to be optimized for best detection (cf "Reglage du modèle")
11. The parameter `multiplier_tresh_accel` is a multiplicative factor applied to the acceleration threshold. For example, a value of 1 means the percentile is used directly. Values above 1 make Wake/REM detection more restrictive, while values below 1 increase sensitivity to movement.
 - a. For now i have tested this parameter to be optimized for best detection (cf "Reglage du modèle")
12. The parameter `saving` determines whether IS and Wake/REM timings are written to disk. When set to True, detected state transitions are stored at the end of the acquisition. When set to False, no file output is produced. This parameter has no influence on detection logic, only on data persistence.
 - a. DEPENDS ON YOU, if you want to save the results of the detection.
 - b. If True, be sure to indicate the path where it could be saved (cf 13.)
13. The parameter `path` defines the directory where output files are saved. For example, `"/media/data-103/Nathan/Avalanche_project/aval_data/"` specifies the storage location. The directory must exist and be writable; otherwise, file saving will fail.
 - a. DEPENDS ON YOU, if you want to save the results (the timings of IS) of the detection.
 - b. SOMETHING GOOD TO KNOW : i have called the file created : `"IS_wake_timings.txt"` (ligne 413 of the code), feel free to rename it on the code (ti change it : cf 14.
 - c. To be sure it will work well, create the file in advance to avoid a bug (happens at the very beginning i don't know why.
14. The parameter `file_name` is the name you choose to the file where the data of timings of the IS and wake/REM will be registered

15. The parameter `mean_spike_rate_channel` defines which channel index is used to read the mean spike rate signal. For example, channel 32 indicates that spike rate values are extracted from that channel in the Open-Ephys signal chain. An incorrect channel selection will result in invalid detection behavior.
 - a. DEPENDS ON YOU, really important to know where you have put the channel (Output of mean spike rate plugin)
16. The parameters `accel_channel_1`, `accel_channel_2`, and `accel_channel_3` define the channel indices corresponding to the three axes of the accelerometer. For example, 33, 34, and 35 indicate the channels used to compute total acceleration magnitude. If these channels are incorrectly assigned, Wake/REM detection will not reflect actual movement.
 - a. DEPENDS ON YOU, really important to know where you have put the channels (position numero in the list of the channel in the channel map, you can change their place in the channel map but you have to change it the numero of their place in this code too as well !)

SAVINGS :

At the end of the acquisition, the timings of IS and Wake/REM periods are saved in a text file named `IS_wake_timings.txt` in the directory defined by the parameter `path`. The file contains the acquisition number, the list of start markers for IS periods, and the list of start markers for Wake/REM periods. (cf PARAMETERS Section).

LAST ADVICES :

- Before starting any acquisition, the selected channels and thresholds should be adjusted if the recording setup differs from the original configuration.
- The TTL panel should be tested to ensure that `ligne_threshold_integration` and `ligne_stop_IS_now` trigger the expected behaviors.
- The saving directory should also be verified to confirm that the program has permission to write the output text file.
- if you do a change in the signal chain, i have always quit (on hubel with ctrl + C) and reopen open-ephys to refresh it (i know that is a shit functionality but that is what has worked until there lol). Because you want to be sure that your changes are taken into account

Reglages du modèle

 IS_detection : Parameters tuning

Advances

OBJECTIF 1 :

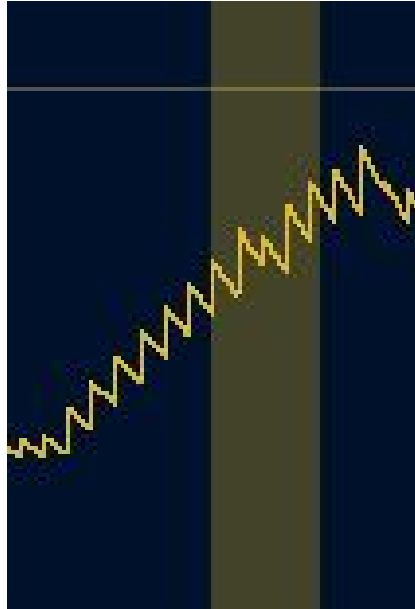
- ☒ Avoir en direct une courbe traçant le nombre de spike au cours du temps s'appuyant sur plusieurs canaux (on additione les spike detected pour ces différents canaux)

Description avancées :

- via un paysage file reader > bandpass filter > spike detector > meanspike rate (choisir le canal pour mettre l'affigage sur :) > LFP viewer
 - o je comprends pas a quoi corresponds les bonnes verticales transparentes rouges (des moments de firing? ca se trouve ca repère des sortes d'avalanches d etoutes la population ?)
 - En fait c'est juste un indicateur d'un événement (TTL) mais ca dépend de ce qu'on a décidé de montrer dans le doc sauvegardé
- on voit une évolution quasi continue ca paraît étrange
 - o choisir les bonnes channels à étudier : demander à Myriam
- on peut récupérer l'information de l'évolution du mean spike rate et utiliser un code python (python processor) pour analyser les données et détecter les moments d'avalanche
 - o demande sur discord si ma méthode est bonne ?
- plusieurs spikent compte plusieurs fois (alors que c'est le meme)
 - ☐ choix de channels loin entre elles
- Pietro conditions :
 - o Pour détecter une avalanche :
 - 1) dépasser un threshold en valeur. Threshold calculé comme $x\%$ de l'intégration du signal sur y minutes.
 - dans son code il prend le 20eme percentile de l'entierté de son signal.
 - 2) Au delà de 20 secondes sans redescend en dessous du threshold : ce n'est pas une avalanche
 - o Pour détecter un slow rythm :
 - mesurer les creux entre le bas et le threshold de l'avalanche : si $\frac{2}{3}$ de ces creux sont suffisamment bas : c'est du slow rhythm. Sinon non.
 - d'autres critère
- Avancé suite à une réponse :
 - on ajoute plus de channels et on modifie le temps dans le mean spike rate :

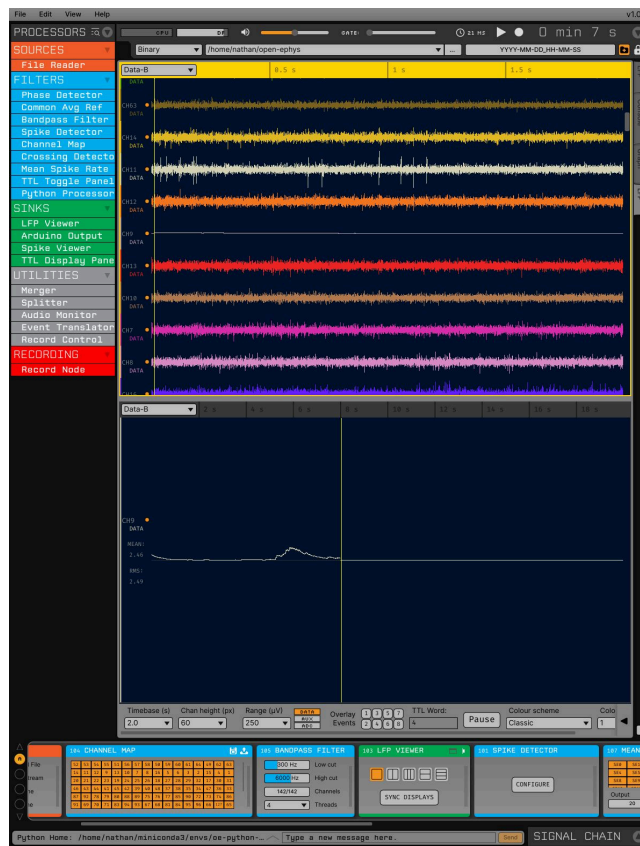


- ajout de crossing detector pour avoir un threshold qui indique un quand on est dans un événement.
 - ☐ régler pour avoir une visualisation des spikes au cours du temps (paramètres de std à régler) qui soit réaliste
 - ☐ comment avoir le threshold qui se règle automatiquement et va directement dans la fonction crossing detector ?
- Questions sur le threshold qui marche pas très bien :



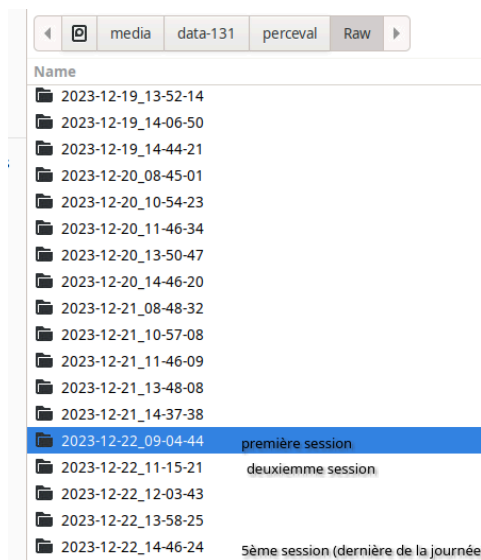
- channel mapping
 - on utilise le fichier xml qui récapitule par rat le channel ajustement (ca se trouve pour un rat à /media/data-131/perceval/Rat003_20231224/Rat003_20231224.xml par exemple) (pas dans Raw)
 - **discuté avec Pascale : avoir un programme**
 - qui prend en argument ce fichier xml.
 - Qui prend les groupes en argument (il y a 6 groupes différents) et les associe à une partie du cerveau. Ex : clu1 : NR clu2: HPC etc...
 - un clusters correspond au bout de la demi probe (il y a deux electrodes par probe)
 - qui renvoie une channel map en fichier .json réarrangé telle que
 - les channles soit dans l'ordre
 - qu'elles soient colorées par cluster
 - petite explication de comment ca se passe en enregistrement :
 - il ya les probe que l'on branche au headstage (il y a un code de pinage ordonné de 1 à 64 si c'est une probe à 64)
 - ensuite le headstage avec l'ampli reschuffle les channels dans un autre ordre (1 de maintenant = le 13 d'avant le headstage, par ex) mais n'indique pas l'ordre (il est prêt à te donner les LFP comme ca
 - DONC il faut ensuite réassocier chaque channel à son pin initial **à la main** pour reconstituer les clusters (les channels de la même demi probe) et **l'indiquer dans un fichier xml**

- **facilitation** : utiliser le site de open-ephys qui registre la plupart des liens entre probe-headstage, et donc avoir le moyen de verifier si on a les bons liens :
<https://open-ephys.github.io/channel-maps-docs/headstages/oeps-6573.html>
- il faut ensuite regarder quand même et verifier que tout est bien arrangé pour conclure le fichier xml.
 - sur quoi Antoine fait -il son xml : à partir de open-ephys et de la channel map réanrragée ?
 - sur neuroscope : lit le fichier xml.
 - sur open-ephys : ne le lit pas donc il faut faire en sorte de lui imposer la channel map du xml.

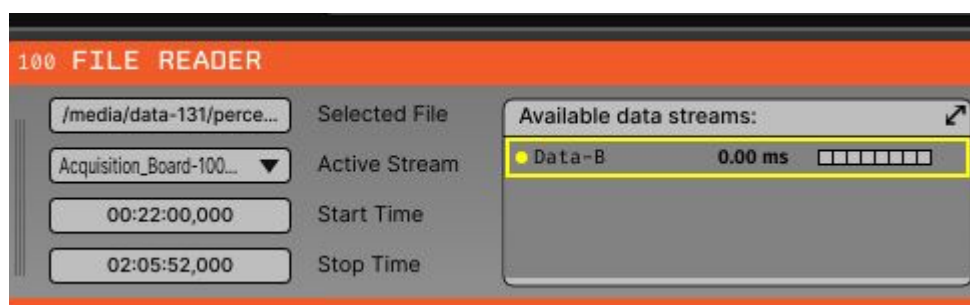


- 19/12/2025 :
 - o code python avalanche_detector fonctionnel : mesure pendant 20 secondes les points du mean spike rate puis prend le xème percentile comme threshold (fixé à 40 pour l'instant)
 - on code sur vs code via : code chemindufichier/monfichier.py
 - o Problème sur la détection du avalanche sur le sleep du réunion : on détecte avalanche que sur le wake (ultra actif) et pas sur le sleep
 - voir si on peut augmenter la sensibilité que pour le sleep ?

- voir si la fenêtre de time value pour le mean spike rate a pas un grand role dans tous ca
- possibilité de modifier les time d'échanges : le buffer est modifiable pour gérer fluidité.
- modification du code matlab pour la **visualisation de la detection des spikes** et des USR par Pietro : on peut choisir lequel des enregistrement dans la journée on veut **1) sleepm 2) tacheem 3) sleepn** (!\ peut ne pas être reconnu sur matlab si le rat ne dort pas) **4) tachea 5) sleeps**
 - tout en gardant le fichier xml principal en session (sleepm et autre deviennent des arguments de regions)
 - exemple pour la journée du 12/22 : on peut apercevoir les 5 sessions (sleepm, tacheem, (sleepn), tachea, sleeps) on les choisit par heure en terme d'ordre (premier à 9h04min44sec)

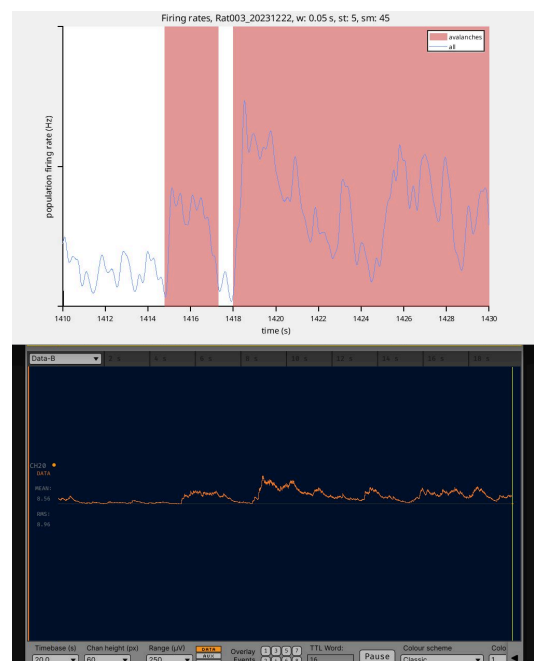
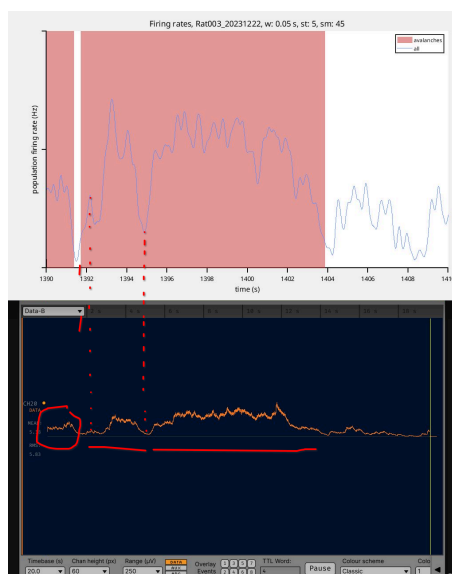
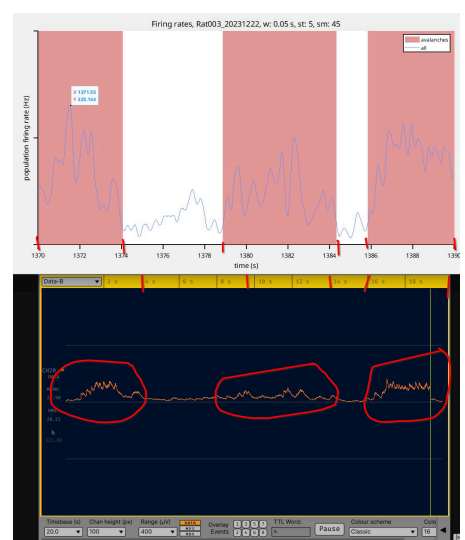
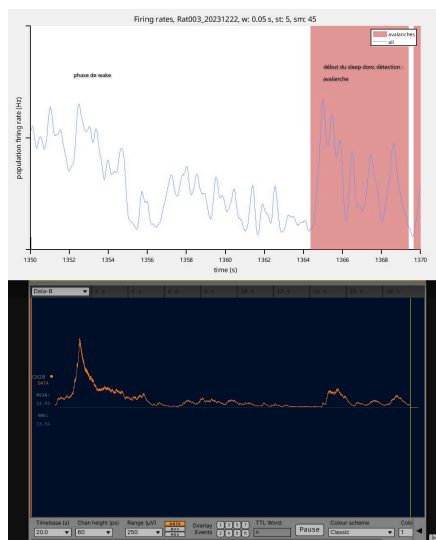


- on peut gérer le temps de début sur openephys (ici commencé à 22 minutes) :
 - dans ce cas il faut refaire CTRLC +
/home/programs/open-ephys/open-ephys : après changement

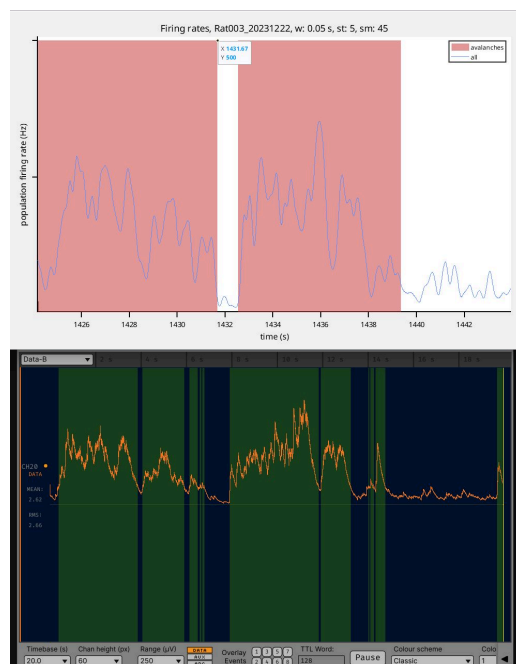
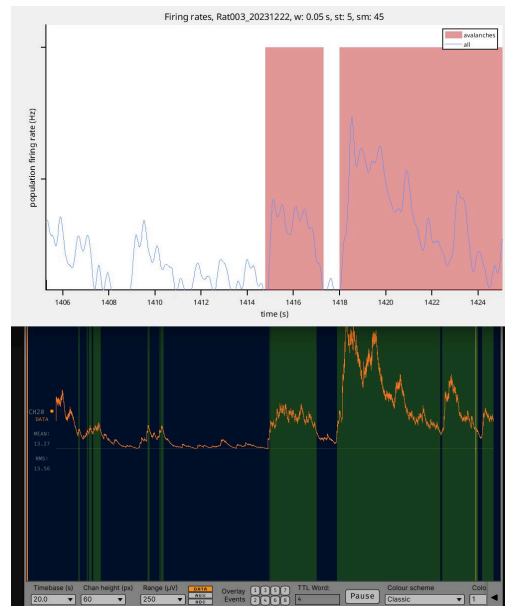
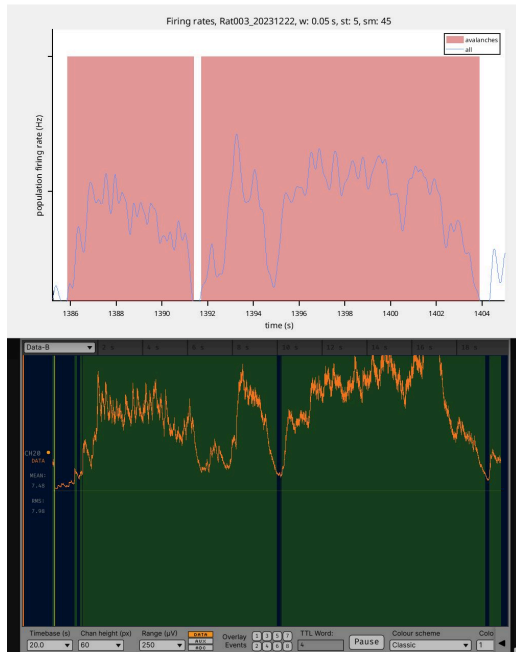


- std 2 trop bas, mdn en dessous de 4 ca fait trop de bruit sinon c'est assez stable

- 13/01/2026 : essayer d'avoir la même forme de firing rate moyen pour wake **et surtout pour sleep**
 - combien de channels avec Réu : 32 donc prendre les premiers paquets
 - Questions avec Pascale :
 - chercher sur internet si multiunit activity se trouve avec le raw signals de toutes les channels et pourquoi pas
 - question de bruits sur toute les channels normal ?
 - Réponses :
 - les gros bruits sont : grincement de dents/se gratte etc.
 - il faudrait se contraindre idéalement sommeil + il bouge pas : voir si les changements ressemble grossièrement.
 - OUI ! résultats sur des phases de dodo pour rat1222 première session à 1350 sec jusqu'à 1390



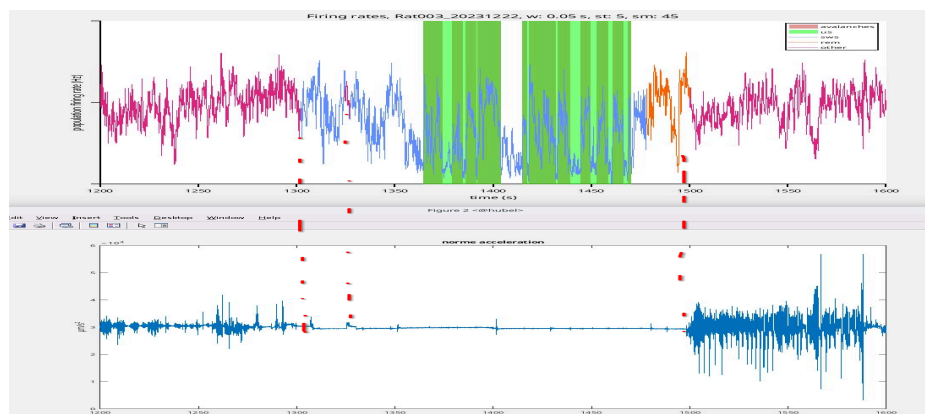
- Résultats avec sleep uniquement et détection des avalanches: YEEAAAA
- changement d'échelle pour la fenêtre du mean firing rate (100 pixels pour mieux voir)
 - bilan : détection marche bien sur le segment de sleep entre 1365 et 1470 sec.
 - **remarque : si c'est wake ou REM : amplitude très forte : je pourrais faire un enregistrement de l'amplitude de qq pics consécutifs et s'ils sont très haut : c'est du wake ou REM !**



- 15/01 : test sur la me^me session mais à 32min40sec
 - marche bien mais détecte plus que Pietro Matlab car prend en compte les zones de fortes activités également (c'est que un treshold pour l'instant)
 - **avoir d'autres critères pour enlever des zones**
 - certaines zones Matlab ne sont pas avalanches je ne sais pas pourquoi (mais détectées par mon programme open-ephys (car threshold))
 - discussion avec Pietro de potentiels critères :
 - critères sur la hauteur (éviter record phase de reveil ou REM) : moyenne ou pic
 - critère sur la longueur des avalanches : imposer un maximum de temps (pour imposer des coupures et éviter des phases hautes)
 - critère sur coupure : imposer un maximum de temps au delà du quel c'est la fin du infraslow
 - critère de début de infraslow : 4 enchainements de coupures succesifs d'avalnaches : infraslow
 - avoir les intervalles de temps par session :
 - si tu changes de sleepm à sleeps ou autre il faut savoir à quel start en secndes c'est possible
 - eventIntervals(R)
- 15/01 : idée de mon programme à mettre dans process: **automate d'état**
 - STEP1 : calcul du threshold (déjà bien fait)
 - STEP2 : calcul des avalanches et du InfraSlow Rythm
 - Dès que avalanche = True :
 - mesure du start time de l'avalanche enregistré dans variable start_time_aval
 - si start_time_cut non négatif :
 - mesure du end time de l'avalanche enregistré dans variable end_time_cut
 - calcul time_cut = end_time_cut - start_time_cut
 - L_time_cuts.append(time_cut)
 - start_time_cut = -1
 - Dès que avalanche = False :
 - si start_time_aval non négatif,
 - mesure du end time de l'avalanche enregistré dans variable end_time_aval
 - calcul de time_aval = end_time_aval - start_time_aval
 - si time_aval > temps_max_aval
 - on réinitialise L_time_cuts = []
 - IS (Infraslow) = False **et** self.processor.add_python_event(6, False)
 - sinon :
 - on autorise à mesurer start_time_cut
 - start_time_aval = -1 (on le réinitialise)
 - Mesure de la moyenne
 - soit on prend le mean(buffer) suffisant ?

- on refait un time counter dont on incrémente la suite continuellement et enlève les premiers éléments. But : qu'il garde une valeur de **time_value_moy = 15 secondes**
 - moy actualisée à chaque buffer
- si moy > **x * threshold_value** :
 - L_time_cuts = []
 - start_time_aval = -1
 - start_time_cut = -1
 - IS = False
- on regarde tous les éléments de L_time_cuts (détection InfraSlow) :
 - si parmi ces temps, chacun est < **max_time_cut**
 - s'il existe au moins 4 de ces éléments qui est > **min_time_cut**
 - IS = True
 - **self.processor.add_python_event(6, True)**
 - mettre des panneaux indicateurs attention on est wake ou REM, self.processor.add_python_event(6, True)
 - **il faut vraiment se dire que les 20 premières secondes doivent être bien choisies sans problème du rat en wake etc**
 - commencer par des paramètres non réducteurs
- 16&17/01 : test des paramètres de mon code pour que ca soit pas trop restrictif
 - test des overlay events (event ttl) : hormis le **self.processor.add_python_event(k, True)** pour k = 2 est déjà occupé! **Le reste (de 0 à 7 sont libres)**
 - "aujourd'hui 2 plots chacun" - Pietro
 - détection et non detection d'IS passable pour session 1360 sec et d'autres sessions
 - remarques sur des points sensibles améliorations du code:
 - le **calcul du trehsold ultra dépendant de où on le lance** et ce pour le reste de l'enregistrement du rat (plusieurs heures)
 - si 20 secondes trop hautes ou trop basses : biase/baise tout
 - comment rendre ce système plus robuste ? fixer à la main le treshold ?
 - moyenne glissante sur moins de 15 secondes pour exclure les pics trop haut (et le critère d'avalanche trop longue prend le dessus), c'est trop
 - réduit à 5 secondes : ca divise un peu plus mes IS ducoup peut etre mettre à 7 ?
 - ajout d'un stop si le cut est trop grand (>10 secondes)
 - pendant le wake ca détecte de l'IS : pas grave qqun est là pour ne pas activer à ce moment là
 - **trouver un moyen d'envoyer les intervalles d'IS (sous forme de Liste par ex pour les comparer avec Matlab à la fin de la session**
 - **en cours avec stop_acquisition il y a un truc a faire**
 - pendant le REM on a des très gros pics puis des plus petits pdt le wake.

- est ce que c'est grave si on détecte pas de l'IS ou on en détecte là où il n'y en a pas ou si il y en a d'après Matlab sur le sleep (annoncé par le code de Marco des intervalles sleep/wake/rem)
- 19/01 et 20/01 :
 - Pietro Marco : veut **utiliser les mesures de l'accéléromètre pour capter si le rat est en wake ou pas**
 - on regarde racine($ax^2 + ay^2 + az^2$) avec un treshhold pour detecter le mouvement
 - on peut pas avoir cette norme en visu open-ephys
 - il faut coder avec ax ay et az les channels 128 - 129 - 130 (en base 0 : ie ca commence à 0 MAIS sur open-ephys c'est compris en commençant par 1 (donc 0 vaut 1 sur open-ephys), ce qui fait 129-130 et 131 sur open-ephys)
 - je les ai mis juste après la chanenel de mean firing rate pour les observer : je vois rien
 - sur matlab on essaye : channels (numero 128,129,130 que j'ai déplacé à la 33 34 et 35e + 1 place)
 - fonctionne plutôt bien si on met un treshhold. (question du pourcentile??)

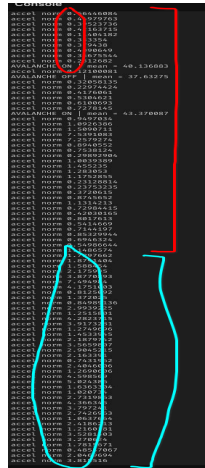


- sur open-ephys je trouve pas les channels associé.
- très faible signal car en e-6 mais open-ephys le lit!
- je teste sur mon code de en print à des frontières entre le sleep et le wake (entre l'inactivité et le mouvement) :

```

accel1 = np.mean(data[33, :])
accel2 = np.mean(data[34, :])
accel3 = np.mean(data[35, :])
norm_acceleration = (accel1**2 + accel2**2 + accel3**2)**(0.5)
print("accel norm",norm_acceleration*10000) #on change juste pour eviter les virgules tout le temps

```



(à25 minutes le changement)en rouge c'est inactif avec début d'activité puis ensuite en bleu c'est la phase de réveil !

- coder la détection d'activité :
 - on fait le calcul comme la moyenne glissante
 - on met un treshold par percentile (**comment l'obtenir sans l'imposer avant d'enregistrer?**)
 - dans un premier temps on associe à un event ttl le mouvement et dans un second temps :
 - on impose un stop du calcul des avalanches et IS si on est dans une phase de mouvement

- trouver un moyen d'enregistrer intervakles d'infrslow mis en pause pour l'instant """" def stop_acquisition(self):


```
print("Saving avalanche data before exit")
```

```

with open("timings_IS.json", "w") as f:
    json.dump(
        self.L_time_IS,
        f,
        indent=2
    )

```

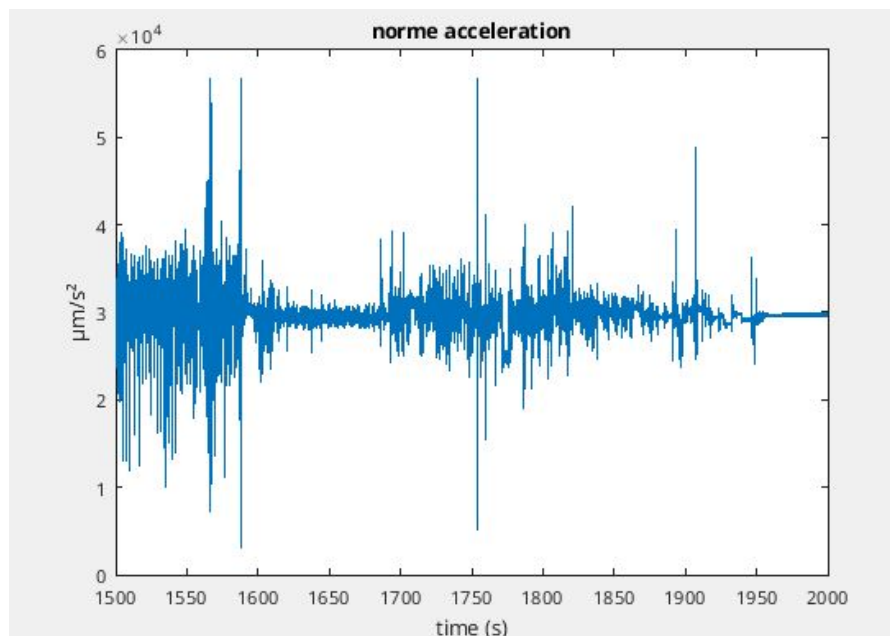
```
print("JSON file written: timings_IS.json")"""
```

- 21/01 : implémentation de l'acceleromètre :

- si crash open-ephys, faire ok avec le message sinon tu vas devoir choisir quelle chaine de signal tu veux restaurer et tu vas devoir la reconstruire d'ailleurs la voici au cas où :



- acceleromètre codage
 - gros beug quand j'implémentais le calcul de l'accélération glissante mauvais code je pense
 - condition de stop car en mouvement : identique à moyenne glissante
 - dangereux les boucles while ?
 - est ce que faire la recuperation de buffer acceleration et en faire direct la moyenne c'est pax ouf ? `accel2 = np.mean(data[34, :])`
 - calcul du treshold :
 - prop1 : on prend sur les 20 premières secondes comme pour les avalanches et dans le nREM (comme avalanche) pour avoir un xièm pourcentage comme treshold.
 - tentative à 2 et 3 x 80eme pourcentage : trop restrictive.
 - tentative au 90eme pourcentage : march ok on dirait
- pb de code is qui veut pas s'arreter à 24min40 + 9min

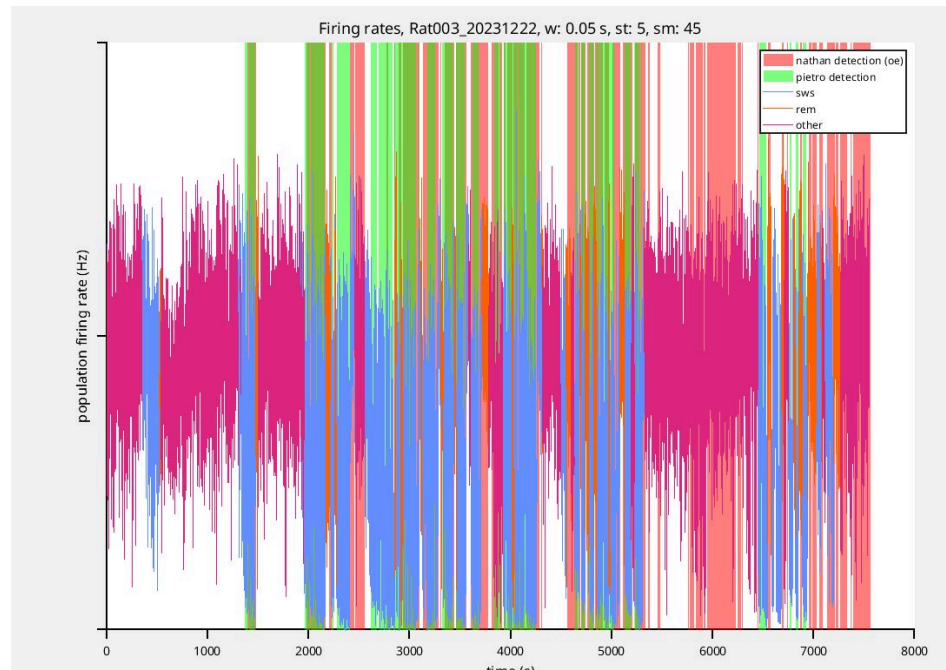


- 22/01 :
 - quelques réflexions du matin:
 - est ce qu'on devrait pas pour l'accélération faire le calcul sur tous les points du buffer plutôt que les moyennes des buffer ?

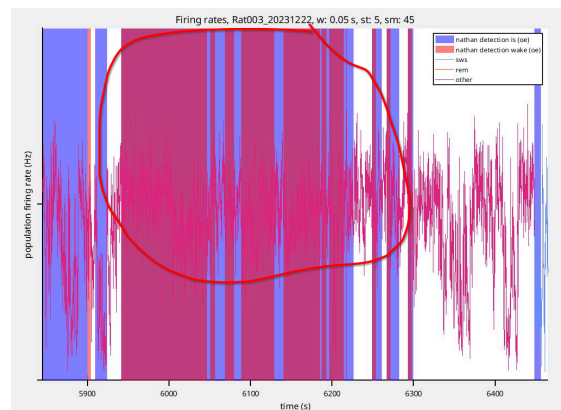
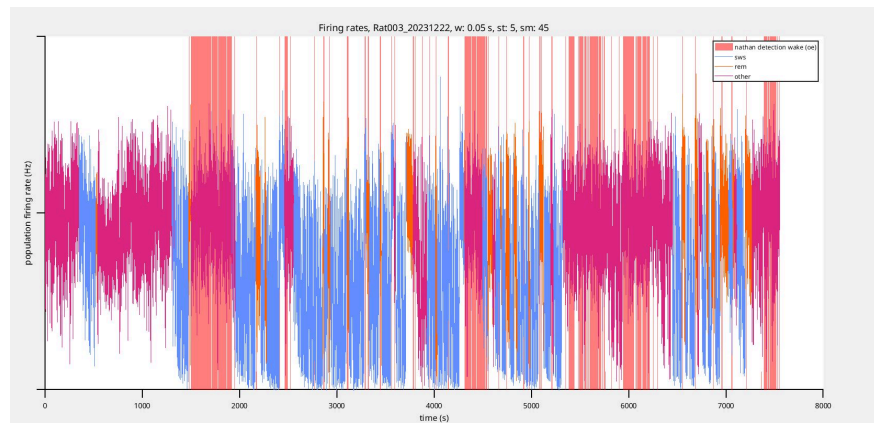
- pour le calcul au bon moment des treshold : avoir une option (avec une seconde branche que l'on peut lancer quand on veut pour lancer le programme). Ex : je regarde le mean firing rate et quand il dort j'envoie l'opération de threshold recording et d'optogenetics en général. A regarder sur open-ephys
 - réponse : oui ! avec le **TTL TOGGLE PANEL** plugin on peut générer soit-même des events par un click. Donc donner un signal au python processor (dans le code) de s'activer et de commencer le calcul des treshold au moment opportun (InfraSlow Rhythm)
- est ce que l'opto peut réveiller le rat :
 - a priori non
- matos pour l'optogénétique
 - arduino uno (alimentation + cable USB pour relier à l'ordinateur) + une LED
 - arduino relié à l'ordinateur avec un code (suivre méthode dans le plugin Arduino Output)
- faire la maj nouvelle version d'open-ephys (avantage pour la lfp)
 - fait !
- faire un algo de comparaison des valeurs de simu versus matlab
 - envoyer une information potentielle a la fin de l'acquisition ?
- test algo IS ailleurs sur la même session
 - recherche sur matlab : idéal 20 sec de IS puis du wake puis du IS à nouveau
 - implémenter la detection wake dans l'algo (faire arreter IS si il est là!)
- 23/01 :
 - calcul du pourcentage mieux avec toutes les acceleration dans le buffer ou pas ?
 - on test sur la fenêtre 1450 s début c'est ok sans faire ça !
 - pb à 5280 marche pas bien...
 -
 - implémentation du wake dans l'algo
 - changement pour difference de valeur absolue du critere pour prendre en compte les oscillations.
 - si on détecte du wake : on mets tout à 0 et on return
 - sinon on met en off
 - pareil que pour la moyenne ? on mets à la fin et si ya juste on réinitialise constamment ?
 - on met moyenne et acceleration comme deux criteres pour voir si on continue à faire detection avalanche et IS ou pas
 - si ya pas alors lets go detection
 - treshold value trop fort ? parametres avalanches longueur et longueur de cut et des moyennes a regler
 - faire enregistrement des intervalles pour pouvoir comparer en fin d'acquisition
 - arduino programme

- 26/01 :
 - intégration creation d'un fichier.txt qui donne la liste de IS timings et des wake_REm time pour pouvoir comparer
 - création d'un fichier .txt
 - pas dans le bon dossier (home/nathan pour l'instant)
 - corrigé avec ption de saving ou pas + mettre le chemin voulu
 - à la fin de l'enregistrement la fin de l'acquisition se stoppe tte seule ?
 - verification : ca repète en boucle
 - **mettre une alarme pour arreter la simulation et appuyer sur stop.**
 - si le fichier n'apparaît pas : il faut stopper plusieurs fois (start/stop) l'acquisition
 - c'est uniquement au moment de sa création (s'il existait pas avant ca galère et ca s'affiche pas puis si tu click plusieurs fois en fait tu réalises qu'il te le met plusieurs fois dans le txt qui vient d'apparaître
 - ca galère quand le fichier est plus lourd
 - relier taggle ttl event pour avoir la la possibilité de lancer analyse / enregistrer les données / relancer la treshold detction.
 - okay je resume mon histoire : j'ai un code ou dans process je crée des boucles if et qui peuvent si verifies envoyer un event `self.processor.add_python_event(3, True)`. Ces events je peux ensuite les voir dns le ttl display panel et egalement sur le lfp viewer quand j'active la bande overlay events attribué à la bonne ligne ttl. Problème j'aimerais avec ttl toggle panel créer un event sur une ligne afin de pouvoir choisir le bon moment pour activer mon programme de detection dy python processor. Cependant ca ne marche pas malgré plusieurs tests : je prit la ligne et le state et il ne recoit rien d'autre qu'un event qui etait deja présent dans l'enregsitrement. j'ai testé de metre ttl toggle panel avant python processor pour l'activer mais ca ne marche pas nn plus (pa setonnant car en soit c'est des buffer donc la chaine de signal boucle) Mais en revanche ttl toggle panel quand je clique ca se detecte ur ttl pdisplay panel.
 - AU FINAL : ca marche ! je peux en mettant bien ttl toggle panel **avant** python processor lui communiquer en direct une information binaire (on off)A revoir mais ca devrait etre bon mais doute
 - ca dpend des canaux de event ttl : le 0 et le 1 et le 3 il arrive pas a envoyer d'évent en mode `sel.processor.add_python_event jsp pq`
- 27/01 :
 - test des ttl toggle panel 👍
 - #la line enclenchée : marche pas pour 0,1,2 | marche pour 3,4,5,6,7,8, 9... un peu près tous sauf 31 par (donc bien tester), ca s'arrête à 31
 - event ttl python : # marche pas pour 0,1,2 | marche pour 3,4,5,6,7
 - mise en place d'une "interface utilisateur avec toggle panel ttl " il nous faut:

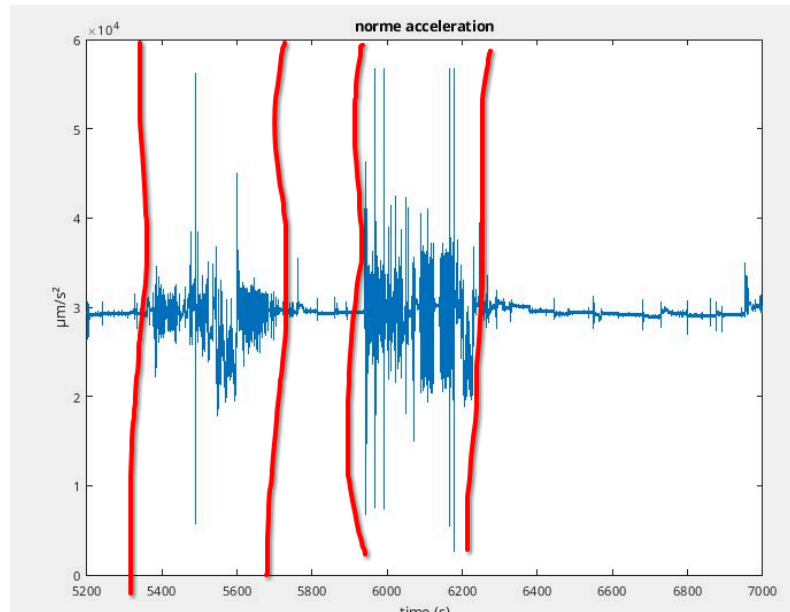
- moyen de déclencher intégration (si True et que avant False : on intègre (on recommence detection si nécessaire en réinitialisant tout SAUF L_time_IS et L_time_wake_REM, mettre tout en False), si False il se passe rien)
- moyen de stopper la IS au cas où (donc stopper la stimulation opto)
 - si True : réinitialisant tout SAUF L_time_IS et L_time_wake_REM, mettre tout en False
- moyen de démarrer IS (donc stimulation)
 - optionnel à discuter
- Discussion avec Marco :
 - faire un **f-score** de mes tests sur 10 heures par exemple. (matrice vrai faux positifs)
 - modèle de machine avec lesquels la arduino va être branché
 - **Cyclops LED Driver**
 - amplificateur pour gérer la puissance qu'on envoie aux LED spécifiques
 - recoit du 3.3 V ou du 5V
 - sortie digitale
 - regarder manuel
 - Au début, on veut que pour IS ca fasse créneau dès qu'on y est puis retombe quand c'est fini.
 - in fine : on veut que pendant cette période on balance des sinus d'une certaine fréquence et amplitude pour stimuler les neurones.
 - peut-être envoyer sur une autre arduino des créneaux d' amplitude et fréquence souhaitées et l'autre arduino transforme ça en sinus ?
 - **Pulse Pal** (modulateur)
 - générateur de signaux
- 2/2/26 :
 - sur le control du panel :
 - quand le user appuie sur toggle panel : il a 2à sec pour retirer le mode On du bouton si on l'integration recommence :
 - Doute : est ce qu'on fait repeter apaaition graphique des states pour éviter de les mettres partout les self.processor.add_python_event ?
 - si user appuie sur bouton stop : arreter la IS detection et réinitialiser tout.
 - pb : quand tu relances il y a plus la detection des IS ??
 - relire code pour voir si ttl event panel marchent bien et sont pas trop répétitif
 - repeter apaaition graphique des states pour éviter de les mettres partout les self.processor.add_python_event
 - test sur les 10 heures de session :
 - on peut mettre sur pause la session
 - liste :
 - session 003_20231222_sleepm (à 9heures) en cours
 - résultat : bcp d'erreur d'IS en trop je dirai. **nécessité de convergence des paramètres vers une détection stable :**



- On voit en rouge des détections trop souvent (Vrais négatifs)
(les paramètres de stop sont trop faibles?)
- analyse des wake intervals détecté :
 - en rouge : les intervalles de wake détecté
 - bizarre à 6000sec il y a du IS mais il y aussi du wake !
ca devrait pas être le cas



- comparaison entre 5000 et 7000 avec acceleromètre :
-



- DONC PROBLEME : LES IS PEUVENT ÊTRE SUPERPOSEES AVEC LES WAKE_REM ?!

- Changement de plan : on reste sur cette session et essaie d'avoir une détection qui soit meilleure :

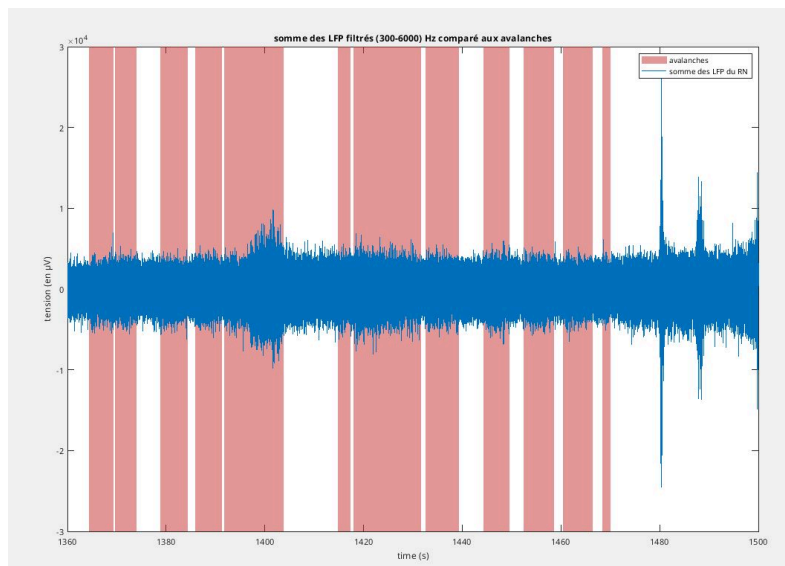
- comparaison : f-score et paramètres tuning :
- **+** IS_detection : Parameters tuning
- Les paramètres pour avoir plus d'IS :
 - diminuer thresh percentile avalanche
- les paramètres pour détecter mieux le wake :
 - diminuer thresh percentile acceleration :
 - diminuer cste multiplicatif pour moy glissante
 - diminuer le temps max d'avalanche
 - augmenter temps de a moy glissante ?
 - augmenter temps moy glissante de l'accélération ?
- erreur corrigée : création d'1 IS si et seulement si il n'y a pas de wake_REM !

- 04/02 :

- questions à étudier, suite à la présentation :
 - **calcul de l'écart type** : savoir comment le spike detector devine l'écart type
 - recherché sur github et demandé à discord
 - Off : 10 secondes (pas assez long?)
 - **On : plus que 12 secondes non ?**
 - **plus discuter de ces paramètres avec Pietro**

- idée Antoine : **sommer les channels filtered et faire un threshold sur la somme des ces channels**
 - idée Christophe : removing all the raw signal with a freq > 1Hz pour voir ce qu'il se passe
 - lancement d'un test d'une autre configuration de paramètres
 - test une autre session open-ephys pour lancer deux tests à la fois
 - deux nouvelles sessions envoyé
 - quelques reflexions :
 - si tu diminues le threshold il faudrait aussi que tes avalanches ON aient une taille minimale et que ca soit pas que sur les cuts !!
 - **etre gentle avec open-ephys en fin de programme car là ca a beugé et je dois recommencer**
 - enregistrer la signal chain plugins ? Pas possible mais tant que tu es sur le open-ephys du serveur hubel tu peux charger plusieurs session x2go client avec le même setup de signal chain.
 - algorithme bash pour extraire d'un fichier xml la bonne channel map:
 - cf plus haut pour les explications.
 - en entrée : fichier xml, extrait
- 05/02:
 - réunion avec Pietro :
 - détection avalanche :
 - Pitero faisait en regardnat les pics les plus bas des passages OFF (possible a faire en prenant le min a chaque fois de chaque buffer)
 - mauvais détection dans des endroits sans acceleration ET peu actif :
 - reduire multiplicator de la moyenne glissante ou reduire
 - réduire constante de temps de la moyenne glissante
 - supprimer les ON trop courts
 - avec verif, **dur de dire si ce sot des réelles avalanche ou pas**
 - à propos des ON trop courts :
 - **si c'est le cas on dit que le OFF est pas terminé et donc : ca divise le nombre de OFF**
 - **en général il faut peut être modifier la methode de calcul des aval et des coupures pour que l'on ai pas besoin de faire le test plusieurs fois !**
 - vérifier que le programmes de statostiques marche vraiment car c'est bizarre...
 - DONE : ca prenait plus en bons intervalles que la session en entier !
 - **pour le test à 1Hz ou pour la somme des channels :**
 - **le faire sur Matlab et regarder les cocordances sur un intervalles avec le On et le Off**
 - **demander à pascalle pour le 1Hz ce qu'elle en pense**
- 10/02 et 11/02 :

- Correction de la fonction de statistiques : j'avais effectivement de meilleurs résultats
 - le pb : les intervalles de us_intervals contenaient plus que les intervalles de la session observée donc j'avais bcp plus !!
- remise en forme du tableau : + IS_detection : Parameters tuning
- changement algorithme méthode de détection On et Off : ce que j'ai modifié :
 - je veux que mon programme detecte la fin d'un OFF et mesure le temps de ce OFF mais n'enregistre rien dans une liste
 - puis au moment de la fin du ON, on mesure le temps de l'avalanche et voit si
 - # 1) aval trop longue : on arrete IS
 - # 2) aval trop petite : on continue le OFF en ignorant cette "fausse avalanche"
 - # 3) aval pas trop grande/petite : on garde la durée du OFF précédente
 - on prend en condition de détection d'IS :
 - aucune coupure trop longue
 - au moins 3 coupures, changement à un min de 0.5 secondes
- lancement de deux sets de paramètres avec une sensibilité plus ou moins grande de min de temps d'avalanche (1 : 57 et 2 : session 58)
- somme des LFP filtré (Christophe/Anton) :
 - comparaison avec les avalanches: rien d'incroyable est percevable en



filtrant toutes les channels puis en les additionnant en en refiltrant le vecteur de la somme..

-
- 12/02 et 13/02 :
 - résultats à l'issu de la nouvelle méthode de detction des OFF avalanches :
 - augmente la precision de 5% pour un time_aval_min de 0,3 secondes MAIS diminue de 8% le recall...
 - observation des résultats graphiquement : cata en recall car j'ai trop diminué le threshold des accelerometres

- j'ai réaugmenté les thresholds accel et j'ai essayé de trouver le bon équilibre (essai 9 et 10 mais ont crashé !!)
 - **résultats** : j'obtiens 60 % de précision et 80% de réussite au mieux pour cette sessions avec les paramètres du test 9
- 16/02 : Arduino time :
 - **résultats** : j'obtiens 60 % de précision et 80% de réussite au mieux pour cette sessions avec les paramètres du test 9
 - tester sur d'autres session avec le même paramétrage ?
 - refaire le test pour être sûr que c'est bien les bons paramètres du test 9 qui donnent ces résultats
 - **tester les spike detector avec médiane**
 - je passe à arduino :
 - télécharger arduino ide (tu extrait + tu donne root pour sandbox)
 - problème sandbox doit être en root
 - firmata pas présent : en fait fallait bien mettre le port et le type de arduino
 - pb de compil : il fallait mettre : `sudo chmod a+rw /dev/ttyACM0` dans un terminal
 - dans ~/arduino/arduino-ide_2.3.7_Linux_64bit faire : `./arduino-ide`
 - `echo 'alias arduino=~/.arduino-ide_2.3.7_Linux_64bit/arduino-ide'`
 - `>> ~/.bashrc`
 - `source ~/.bashrc`
 - permet de juste ercire arduino sur le terminal
 - tentative de mettre en detection le arduino avec le serveur :
 - problème mtn j'arrive plus a ouvrir open-ephys
 - arduino : essayer de compiler avec modulation de fréquence
 - on oet compiler que un code (qui va dans le fichier executable) 1 par 1
 - c'est quoi .ino .h .c ? différence
 - ino c'est des scripts Arduino avec une boucle loop
 - les autres c'est prog en C juste des fonctions ou des class apelable avec include
 - modification du programme FirmataStandard pour inclure une fonction en C qui puisse faire une activation en pulse de ma LED. Connaître la tension
 - ca marche
- 19/02 :
 - moyen de relier arduino avec generateur de sinus
 - **en fait pas besoin de faire clignoter la led juste tu lui donnes StandardFirmata** et ca fait du HIGH quand il faut et du LOW quand il faut
 - relier a un générateur plusieurs possibilités :
 - relier au générateur avec AM (amplitude modulation)
 - interrupteur analogique pour laisser passer sinus ou non
 - multiplueur du arduino avec générateur ?

- ou Cyclops fait peut être déjà ça ?
 - En attendant mes expériences je lance de nouvelles sessions !
 - même rat mais sleeps : **PB ca ne detecte pas bien !!!**
 - dû à la selection du départ (au début pas bon : j'ajoute 40 secondes après le vrai départ (le départ officiel) ca marche très bien)
 - lancement sleeps 20231222 : départ 22min48sec après départ
 - à ajouter au tableau excel
 - mesure sur oscilloscope arduino envoie des créneaux de 0/5v avec le pin et StandardFirmata
- 20/02 :
 - code cleaning Sara
 - bonne session de rat à partir de 1222 pour perceval
 - résultats nouvelles session Session Rat 003_20231212 _ sleeps (à 9h)
 - Cyclops Led driver : comment faire :
 - **Open Ephys → Arduino (StandardFirmata)** Tu envoies des **events TTL** depuis Open Ephys. Arduino (StandardFirmata) reçoit ces events et les traduit en **0 V / 5 V** sur un pin numérique (HIGH = 5 V, LOW = 0 V).
Arduino → Teensy dans Cyclops Ce TTL 0/5 V est connecté sur le **TRIG** du Cyclops.
Teensy embarqué dans le Cyclops lit ce TRIG comme un déclencheur.
 - **Teensy (Cyclops) → DAC(Digital-to-Analog Converter) interne** Tu as uploadé un **sketch Teensy** qui définit : `generate_sine()` → génère un tableau DAC 12 bits pour la sinusoïde
`stop_wave()` → met la sortie à 0 V quand TTL LOW
Teensy exécute la sinusoïde tant que TTL HIGH et stoppe quand LOW.
 - **DAC → Cyclops LED Driver → LED** Le DAC 0–5 V est utilisé par le Cyclops pour moduler le courant dans la LED. La sortie courant/voltage est gérée par le Cyclops, qui applique le signal sinusoïdal sur la LED en respectant la limite de courant définie.
 - Donc programmation du Teensy
 - **cable micro-USB**
 - **LED CURR** : donne une tension proportionnelle au courant réel passant dans la LED (1 V/A).
REF OUT : donne la tension de référence utilisée par le Cyclops. **AUX OUT** : version bufferisée du signal auxiliaire. Ça sert **pour brancher un oscilloscope ou un acquisitionneur** pour vérifier :
- 23/02 :
 - 4 sessions envoyés (4 sur rat perceval et demain sur autre rat: rat 004 sur hubel 139 qui se nomme karadoc)
 - changement de rat : problème : trop de spikes courbe hyper hautes !!
 - juste change l'échelle (range) de la fenêtre
 - **changer le channel mapping !!**

- code de teensy :
 - avoir teensy sur arduino : suivre guide page
https://www.pjrc.com/teensy/td_download.html
 - importer les librairies cyclops de l'adresse :
<https://github.com/jonnew/cyclops/tree/master/lib/cyclops/src>
 - pas oublier que tu mets cyclops.h et cpp dans le m^me dossier que ton code .ino
- j'envoi de la tension qui oscille en sinus de 0 à 4095 bit (0-5 volts normalement !!?)
 - courant suit d'après la doc la fonction

2. **EXT**  **DAC**: External input mode is engaged when the **INPUT SELECT** switch is moved to the **EXT** position and user supplied voltage waveforms are present at the **EXT BNC** input. If the user attempts to supply more than 5V to the **EXT** input, the circuit will clamp the input signal to 5V. Using **EXT** mode, the behavior of the circuit is:

- **AUX**  **CURR**: Source the current specified by:

$$\text{Load Current} = \text{GAIN Position} * \frac{\text{SIGNAL IN Voltage}}{5V}$$

- **AUX**  **CURR**: Generate the optical power specified by:

$$\text{Power} = \text{GAIN Position} * h * \frac{\text{SIGNAL IN Voltage}}{5V}$$

The intensity of the LED will be dependent on the auxiliary feedback gain which defines the transmittance

- Par rapport à **Pulse Pal** :
 - intéressant si on veut envoyer des pulses mais pour l'instant mon montage fait l'affaire

TO DO LIST

TO DO List :

- ☒ ~~verifier branchements expérence Antoine~~
 - ☒ ~~savoir avec quoi il fait les fichiers xml (demander à Antoine et Myriam)~~
- ☐ programme de traduction des fichiers xml en channel mapping
- ☒ regarder comment marche le python process plugin
 - ☒ ~~verifier que l'on peut bien prendre un input en continue et envoyer un output en continue:~~
- ☒ ~~voir si on peut augmenter la sensibilité que pour le sleep ? du std (superieure à 2 ou 3)~~
- ☒ ~~voir si la fenêtre de time value pour le mean spike rate a pas un grand role dans tous ça~~
- ☒ objectif 1 : avoir une courbe de mean spike rate semblable à la courbe de matlab
 - ☒ ~~question de buffer ?~~
 - ☒ ~~question de constante de temps ? (sensibilité de la détection)~~
- ☐ threshold value trop fort ? parametres avalanches longueur et longueur de cut et des moyennes a regler
- ☒ ~~faire enregistreemnt des intervalles pour pouvoir comparer en fin d'acquisition :~~
 - ☐ je sais mtn comment enregistrer et comparer les données
- ☐ lancer les mesures sur les autres sessions :
- ☐ converger sur les bons paramètres :
 - ☐ f-core
 - ☐ avec les paramètres affichés
- ☐ arduino programme

Onglet 4

```

import numpy as np
import math
import oe_pyprocessor
from collections import deque # utile pour la moyenne glissante
# On importe la classe :
class PyProcessor:

    def __init__(self, processor, num_channels, sample_rate):
        """
        A new processor is initialized whenever the plugin settings are updated

        Parameters:
        processor (object): Python Processor class object used for adding events from python.
        num_channels (int): number of input channels in the selected stream.
        sample_rate (float): sample rate of the selected stream
        """
        print("Num Channels: ", num_channels, " | Sample Rate: ", sample_rate)
        # On récupère les informations données par Open-ephys
        self.processor = processor
        self.num_channels = num_channels
        self.sample_rate = sample_rate
        # Nouveau arguments :
        self.time_counter = 0.0
        self.time_limit = 20.0 # fixé à 20 secondes par défaut pour l'instant
        self.buffer_points = []
        self.percentage = 70 # fixé à 40eme pourcentage pour percentile
        self.threshold_go = False
        self.threshold_value = None
        self.avalanche_state = False # état courant (OFF au départ)

        #les events :
        self.ttl_aval = 7          # ligne TTL utilisée
        self.ttl_IS = 4

        # les temps d'aval et de coupures etre avalanche
        self.start_time_aval = -1
        self.end_time_aval = 0
        self.start_time_cut = -1
        self.end_time_cut = 0

        # liste qui enregistre les coupures :
        self.L_time_cuts = []

        # booleen de l'InfraSlow Rythm :
        self.IS = False

        # parameters of limitations :

```

```

self.max_time_aval = 20 # temps d'avalanche max
self.time_value_moy = 15 # temps du calcul de la moyenne
self.multiplier_tresh = 3 # constante multiplicative evaluation de moy >
multiplier_tresh.threshold_value ou pas
self.max_time_cut = 10 # temps max d'une coupure
self.min_time_cut = 0.1 # temps min d'au moins 4 coupures parmi toutes

# pour la moyenne :
self.mean_buffers = deque()      # stocke (time, value)
self.mean_buffer_time = 0.0

pass

def process(self, data):
    """
    Process each incoming data buffer.

    Parameters:
    data - N x M numpy array, where N = num_channels, M = num of samples in the buffer.
    """
    # ETAPE 0 : recupere datas + calcul du time_counter et de la moyenne instantanee et
    glissante :

    # récupération de la channel du mean_spike_rate
    mean_spike = data[32, :] # a été fixé à la 33eme channel (MODIFIABLE)

    buffer_duration = mean_spike.shape[0] / self.sample_rate # duree de ce buffer
    self.time_counter += buffer_duration # on l'ajoute à notre counter

    ## calcul (maj) de la moyenne glissante :

    # moyenne du buffer courant : peut être à faire avec le min mais moins robuste
    current_mean = np.mean(mean_spike)
    # a present calcul de la moyene glissante :
    self.mean_buffers.append((buffer_duration, current_mean))
    self.mean_buffer_time += buffer_duration

    # on enleve les valeurs trop anciennes et ajout des nouvelles pour qu'on reste sur un
    temps de time_value_moy maximal
    while self.mean_buffer_time > self.time_value_moy:
        dt, val = self.mean_buffers.popleft()
        self.mean_buffer_time -= dt

    # on moyennise :
    moy_glissante = np.mean([v for _, v in self.mean_buffers])

```

```

        #On ajoute les valeurs du buffer tant qu'on est en dessous de la limite
integration_total_time
        # ETAPE 1 : CALCUL DU TRESHOLD
        if not self.threshold_go:

            self.buffer_points.append(mean_spike) # on accumule les valeurs du buffer dans la
liste buffer_points

        if self.time_counter >= self.time_limit:
            self.threshold_go = True

            #METHODE 1 (Pietro) : On prend la valeur du xème percentile
            all_values = np.concatenate(self.buffer_points) # on concatene la liste de listes
de valeurs
            self.threshold_value = np.percentile(all_values, self.percentage) # calcul du
pourcentage qui sera notre threshold
            #self.processor.add_python_event(5, True) # regarder output named "6" pour voir
que c'est bon on a le calcul du threshold

        return #on ne détecte pas encore les avalanches

# ETAPE 2 : CALCUL DES AVALANCHES ET DE L'INFRASLOW :

# décision avalanche
avalanche_now = current_mean > self.threshold_value

## transition OFF -> ON (presence avalanche)
if avalanche_now and not self.avalanche_state:
    self.avalanche_state = True
    print("AVALANCHE ON | mean =", current_mean)

#calcul du temps de début de l'avalanche :
self.start_time_aval = self.time_counter
#calcul du dernier cut (espace entre deux avalanches) et ajout dans L_time_cuts :
if self.start_time_cut >= 0:
    self.end_time_cut = self.time_counter
    time_cut = self.end_time_cut - self.start_time_cut
    self.L_time_cuts.append(time_cut)
    #réinitialisation du début de cut :
    self.start_time_cut = -1

# affichage de l'avalanche :
self.processor.add_python_event(self.ttl_aval, True)

## transition ON -> OFF (fin avalanche)

```

```

elif not avalanche_now and self.avalanche_state:
    self.avalanche_state = False
    print("AVALANCHE OFF | mean =", current_mean)

# on mesure le temps de l'avalanche
if self.start_time_aval >= 0:
    self.end_time_aval = self.time_counter
    time_aval = self.end_time_aval - self.start_time_aval
    # si c'est trop grand on est plus dans sleep donc on arrete tout
    if time_aval > self.max_time_aval:
        self.L_time_cuts = []
        self.IS = False
        self.processor.add_python_event(self.ttl_IS, False)
    # sinon on continue en commençant a mesurer la coupure (cut)
    else :
        self.start_time_cut = self.time_counter

# affichage fin de l'avalanche :
self.processor.add_python_event(self.ttl_aval, False)

## condition sur la moyenne (eviter cas REM ou wake) :
# ne pas s'engager tant qu'on est inferieur à la milimite de calcul de moyenne
if self.mean_buffer_time < self.time_value_moy:
    return
# test moyenne suffisamment basse (sinon on recommence)
if moy_glissante > self.multiplier_tresh * self.threshold_value :
    self.L_time_cuts = []
    self.start_time_aval = -1
    self.start_time_cut = -1
    self.IS = False
    self.processor.add_python_event(self.ttl_IS, False)

## detection InfraSlow Rythm (IS) :
if len(self.L_time_cuts) >= 4:

    # condition 1 : aucune coupure trop longue
    cond_max = all(t < self.max_time_cut for t in self.L_time_cuts)

    # condition 2 : au moins 4 coupures suffisamment longues
    long_cuts = [t for t in self.L_time_cuts if t > self.min_time_cut]
    cond_min = len(long_cuts) >= 4

    if cond_max and cond_min and not self.IS:
        self.IS = True
        self.processor.add_python_event(self.ttl_IS, True)

return

```

```

self.time_value_moy_acceleration = 6 #
# pour l'acceleration (moyenne glissante)
    self.moy_acceleration_buffers = deque() #stocke (time,value)
    self.moy_acceleration_buffer_time = 0.0

## calcul (maj) de la moy acceleration glissante :
    # moy instant acceleration :
    norm_acceleration = (accel1**2 + accel2**2 + accel3**2)**(0.5)
    #print("accel norm",norm_acceleration*10000) #on change juste pour eviter les virgules
tout le temps
    # a present calcul de la moyenne glissante acceleration:
    self.moy_acceleration_buffers.append((buffer_duration, norm_acceleration))
    self.moy_acceleration_buffer_time += buffer_duration
    # on enleve les valeurs trop anciennes et ajout des nouvelles pour qu'on reste sur un
temps de time_value_moy maximal
    while self.moy_acceleration_buffer_time > self.time_value_moy_acceleration:
        dt, val = self.moy_acceleration_buffers.popleft()
        self.moy_acceleration_buffer_time -= dt
    # on moyennise :
    moy_acceleration_glissante = np.mean([v for _, v in self.moy_acceleration_buffers])
    #print("accel norm moy glissante",moy_acceleration_glissante*10000) #on change
juste pour eviter les virgules tout le temps

```

```

import numpy as np
import math
import oe_pyprocessor
from collections import deque # utile pour la moyenne glissante
import json
# On importe la classe :
class PyProcessor:

```

```

    def __init__(self, processor, num_channels, sample_rate):
        """

```

A new processor is initialized whenever the plugin settings are updated

Parameters:

processor (object): Python Processor class object used for adding events from python.

num_channels (int): number of input channels in the selected stream.

sample_rate (float): sample rate of the selected stream

```
        """
```

```
        print("Num Channels: ", num_channels, " | Sample Rate: ", sample_rate)
```

```
        # On récupère les informations données par Open-ephys
```

```
        self.processor = processor
```

```
        self.num_channels = num_channels
```

```
        self.sample_rate = sample_rate
```

```

        # General et calcul Etape 1 calcul thresholds:

```

```

self.time_counter = 0.0
self.buffer_points = []
self.accel_points = []
self.threshold_value = None
self.threshold_acceleration = None

# boolens/etats :
self.threshold_go = False
self.avalanche_state = False # état courant (OFF au départ)

#les events :
self.ttl_aval = 7          # ligne TTL utilisée pour etat avalanche
self.ttl_IS = 4            # ligne TTL utilisée pour etat InfraSlow
self.ttl_wake = 1          # ligne TTL utilisée pour etat Wake

# les temps d'aval et de coupures etre avalanche
self.start_time_aval = -1
self.end_time_aval = 0
self.start_time_cut = -1
self.end_time_cut = 0

# listes qui enregistrent les coupures et les IS timings :
self.L_time_cuts = []
self.L_time_IS = []

# booleen de l'InfraSlow Rythm :
self.IS = False

# parameters of limitations :
self.time_limit_integration = 20.0 # fixé à 20 secondes par défaut pour l'instant
self.percentage = 70 # fixé à 70eme pourcentile pour percentile du treshold des valeurs
mean firing rate
self.percentage_accel = 90 # xièm pourcentile des valeurs du vecteur des valeurs de
la norm de l'acceleration
self.max_time_aval = 20 # temps d'avalanche max
self.time_value_moy = 7 # temps du calcul de la moyenne glissante (sa taille de
fenêtre)
self.time_value_acceleration = 6 # temps du calcul de la moyenne glissante de
l'acceleration (sa taille de fenêtre)
self.multiplier_tresh = 3 # constante multiplicative evaluation de moy >
multiplier_tresh.threshold_value ou pas
self.multiplier_tresh_accel = 1 # constante multiplicative pour threshold acceleration
self.max_time_cut = 10 # temps max d'une coupure
self.min_time_cut = 0.1 # temps min d'au moins 4 coupures parmi toutes

# pour la moyenne glissante:
self.mean_buffers = deque()      # stocke (time, value)

```

```

self.mean_buffer_time = 0

# pour l'acceleration glissante:
self.acceleration_buffers = deque() #stocke (time,value)
self.acceleration_buffer_time = 0

pass

def process(self, data):
    """
    Process each incoming data buffer.

    Parameters:
    data - N x M numpy array, where N = num_channles, M = num of samples in the buffer.
    """

    # ETAPE 0 : recupere datas + calcul du time_counter + la moyenne instantanee et
    glissante + moyenne acceleration instant et glissante:

    # récupération de la channel du mean_spike_rate
    mean_spike = data[32, :] # a été fixé à la 33eme channel (MODIFIABLE)
    # recuperation des channels de l'accelerometre, place juste apres le mean firing rate (/!\
    risque de chgt en fonction du xml)
    accel1 = np.mean(data[33, :])
    accel2 = np.mean(data[34, :])
    accel3 = np.mean(data[35, :])

    # calcul du temps :
    buffer_duration = mean_spike.shape[0] / self.sample_rate # duree de ce buffer
    self.time_counter += buffer_duration # on l'ajoute à notre counter

    ## calcul (maj) de la moyenne glissante :
    # moyenne du buffer courant : peut être à faire avec le min mais moins robuste
    current_mean = np.mean(mean_spike)
    # a present calcul de la moyene glissante :
    self.mean_buffers.append((buffer_duration, current_mean))
    self.mean_buffer_time += buffer_duration
    # on enleve les valeurs trop anciennes et ajout des nouvelles pour qu'on reste sur un
    temps de time_value_moy maximal
    while self.mean_buffer_time > self.time_value_moy:
        dt, val = self.mean_buffers.popleft()
        self.mean_buffer_time -= dt
    # on moyennise :
    moy_glissante = np.mean([v for _, v in self.mean_buffers])

    ##calcul (maj) de la moy acceleration glisaante :
    #moy instant acceleration :

```

```

    norm_acceleration = ((accel1**2 + accel2**2 + accel3**2)**(0.5))*10000 # on multiplie
par 10000 pour plus de lisibilite
    print("norm acceleration", norm_acceleration)
    # a present calcul de la moyenne glissante acceleration:
    self.acceleration_buffers.append((buffer_duration, norm_acceleration))
    self.acceleration_buffer_time += buffer_duration
    # on enleve les valeurs trop anciennes et ajout des nouvelles pour qu'on reste sur un
temps de time_value_moy maximal
    while self.acceleration_buffer_time > self.time_value_acceleration:
        dt, val = self.acceleration_buffers.popleft()
        self.acceleration_buffer_time -= dt
    # on moyennise :
    acceleration_glissante = np.mean([v for _, v in self.acceleration_buffers])
    print("accel norm moy glissante", acceleration_glissante) #on change juste pour eviter
les virgules tout le temps
    print("threshold acceleration", self.threshold_acceleration)

    #On ajoute les valeurs du buffer tant qu'on est en dessous de la limite
integration_total_time
    # ETAPE 1 : CALCUL DU TRESHOLD
    if not self.threshold_go:
        #accumulation des valeurs du buffer de mean_firing_rate :
        self.buffer_points.append(mean_spike)
        # accumulation des valeurs de l'acceleration dans ce vecteur
        self.accel_points.append(norm_acceleration)

        # si fin de la periode d'integration (x secondes)
        if self.time_counter >= self.time_limit_integration:
            self.threshold_go = True #on a calcule les thresholds

            # METHODE : On prend la valeur du xème percentile
            all_values = np.concatenate(self.buffer_points) # on concatenate la liste de listes
de valeurs
            self.threshold_value = np.percentile(all_values, self.percentage) # calcul du
pourcentile qui sera notre threshold

            # application de la eme methode pour le treshold d'acceleration:
            # pas besoin de concatener car deja dans la bonne forme
            self.threshold_acceleration = np.percentile(self.accel_points,
self.percentage_accel) # calcul du pourcentile qui sera notre threshold
            return #on n'engage pas la suite de la detection

    # ETAPE 2 : CALCUL DES AVALANCHES ET DE L'INFRASLOW :

    # detection du wake :
    condition_wake = (acceleration_glissante > self.threshold_acceleration *
self.multiplier_tresh_accel)

```

```

if condition_wake:
    self.processor.add_python_event(self.ttl_wake, True)
elif not condition_wake :
    self.processor.add_python_event(self.ttl_wake, False)

## condition de STOP sur les cuts trop grands:
cond_max_cut = (self.time_counter - self.start_time_cut < self.max_time_cut)
if self.IS and self.start_time_cut >= 0 and not cond_max_cut:
    self.IS = False
    L_time_cuts = []
    #on reinitialise aussi le cut pour pas qu'il soit pris en compte dans la prochaine serie
de cuts entre avalanche
    self.start_time_cut = -1
    self.processor.add_python_event(self.ttl_IS, False)

# décision avalanche
avalanche_now = current_mean > self.threshold_value

## transition OFF -> ON (presence avalanche)
if avalanche_now and not self.avalanche_state:
    self.avalanche_state = True
    print("AVALANCHE ON | mean =", current_mean)

#calcul du temps de début de l'avalanche :
self.start_time_aval = self.time_counter
#calcul du dernier cut (espace entre deux avalanches) et ajout dans L_time_cuts :
if self.start_time_cut >= 0:

    self.end_time_cut = self.time_counter
    time_cut = self.end_time_cut - self.start_time_cut
    self.L_time_cuts.append(time_cut)
    #réinitialisation du début de cut :
    self.start_time_cut = -1

# affichage de l'avalanche :
self.processor.add_python_event(self.ttl_aval, True)

## transition ON -> OFF (fin avalanche)
elif not avalanche_now and self.avalanche_state:
    self.avalanche_state = False
    print("AVALANCHE OFF | mean =", current_mean)

# on mesure le temps de l'avalanche
if self.start_time_aval >= 0:
    self.end_time_aval = self.time_counter
    time_aval = self.end_time_aval - self.start_time_aval
    # si c'est trop grand en temps on n'est plus dans sleep donc on arrete tout

```

```

if time_aval > self.max_time_aval and self.IS:

    self.L_time_cuts = []
    self.IS = False
    #enregistrement de la fin du IS
    self.L_time_IS.append(self.time_counter)
    self.processor.add_python_event(self.ttl_IS, False)
    #si on est pas dans IS mais que j'ai un trop grand on réinitialise tout
    if time_aval > self.max_time_aval and not self.IS:

        self.L_time_cuts = []
        self.IS = False
        self.processor.add_python_event(self.ttl_IS, False)

    # sinon on continue en commençant a mesurer la coupure (cut)
    else :
        self.start_time_cut = self.time_counter

# affichage fin de l'avalanche :
self.processor.add_python_event(self.ttl_aval, False)

## condition sur la moyenne (eviter cas REM ou wake) :
"""# ne pas s'engager tant qu'on est inferieur à la milimite de calcul de moyenne
# mis en comme,ntaire car trop restrictif ca marchait jamais
if self.mean_buffer_time < self.time_value_moy:

    return""""

# test moyenne suffisamment basse (sinon on recommence)
if moy_glissante > self.multiplier_tresh * self.threshold_value and self.IS:
    self.L_time_cuts = []
    self.start_time_aval = -1
    self.start_time_cut = -1
    self.IS = False
    self.L_time_IS.append(self.time_counter)
    self.processor.add_python_event(self.ttl_IS, False)

#print("liste IS intervals: ",self.L_time_cuts)

## detection InfraSlow Rythm (IS) :
if len(self.L_time_cuts) >= 4 :

    # condition 1 : aucune coupure trop longue
    cond_max = all(t < self.max_time_cut for t in self.L_time_cuts)

    # condition 2 : au moins 4 coupures suffisamment longues
    long_cuts = [t for t in self.L_time_cuts if t > self.min_time_cut]

```

```
cond_min = len(long_cuts) >= 4
```

```
if cond_max and cond_min:  
    #print("max liste cuts: ", np.max(self.L_time_cuts))  
    self.IS = True  
    self.processor.add_python_event(self.ttl_IS, True)  
    self.L_time_IS.append(self.time_counter)
```

```
return self.L_time_IS
```

```
import numpy as np  
import math  
import oe_pyprocessor  
from collections import deque # utile pour la moyenne glissante  
import os  
import fileinput  
# On importe la classe :  
class PyProcessor:
```

```
def __init__(self, processor, num_channels, sample_rate):  
    """
```

A new processor is initialized whenever the plugin settings are updated

Parameters:

processor (object): Python Processor class object used for adding events from python.

num_channels (int): number of input channels in the selected stream.

sample_rate (float): sample rate of the selected stream

```
    """
```

```
print("Num Channels: ", num_channels, " | Sample Rate: ", sample_rate)
```

```
# On récupère les informations données par Open-ephys
```

```
self.processor = processor
```

```
self.num_channels = num_channels
```

```
self.sample_rate = sample_rate
```

```
# General et calcul Etape 1 calcul thresholds:
```

```
self.time_counter = 0.0
```

```
self.buffer_points = []
```

```
self.accel_points = []
```

```
self.threshold_value = None
```

```
self.threshold_acceleration = None
```

```
self.mean_acceleration_decay = None
```

```
# pour la moyenne glissante:
```

```

self.mean_buffers = deque()      # stocke (time, value)
self.mean_buffer_time = 0

# pour l'acceleration glissante:
self.acceleration_buffers = deque() #stocke (time,value)
self.acceleration_buffer_time = 0

# boolens/etats :
self.threshold_go = False      # etat on a fini le calcul des treshold sur les x secondes
nécessaires
self.avalanche_state = False # etat ON ou OFF avalanche (OFF au départ)
self.IS = False                # etat ON ou OFF InfraSlow Rhythm (OFF au départ)
self.wake_REM = False          # etat ON ou OFF Wake ou REM (OFF au départ)

#les events de python ttl :
self.ttl_aval = 7              # ligne TTL utilisée pour etat avalanche
self.ttl_IS = 4                # ligne TTL utilisée pour etat InfraSlow
self.ttl_wake = 1              # ligne TTL utilisée pour etat Wake

# la communication avec utilisateur (ttl toggle panel), /\ numero a cliquer sur TTPanel :
valeur + 1 :
self.ligne_threshold_integration = 9 # pour engager le calcul du threshold
self.ligne_redo_tresh = 28          # pour REengager le calcul du threshold
self.ligne_stop_IS_now = 14         # pour arreter stimulation et remettre à 0
IS/avalanche states
self.user_thresh_go = False          # etat associe a la ligne
ligne_threshold_integration
self.user_redo_tresh = False          # etat associe au premier lancement du treshhold
self.user_stop_IS_now = False        # etat associe a la ligne ligne_stop_IS_now

# les temps d'avallanches (aval) et de coupures (cuts) etre avalanche
self.start_time_aval = -1
self.end_time_aval = 0
self.start_time_cut = -1
self.end_time_cut = 0

# listes qui enregistrent les coupures, les IS timings et les wake_REM timings :
self.L_time_cuts = []
self.L_time_IS = []
self.L_time_wake_REM = []

# parameters of limitations : possible de jouer sur ces parametres
self.time_limit_integration = 20.0 # fixé à 20 secondes par défaut pour l'instant
self.percentage = 70 # fixé à 70eme pourcentile pour percentile du treshhold des valeurs
mean firing rate
self.percentage_accel = 90 # xièm pourcentile des valeurs du vecteur des valeurs de
la norm de l'acceleration

```

```

self.max_time_aval = 12 # temps d'avalanche max
self.time_value_moy = 5 # temps du calcul de la moyenne glissante (sa taille de
fenêtre)
self.time_value_acceleration = 5 # temps du calcul de la moyenne glissante de
l'accélération (sa taille de fenêtre)
self.multiplier_tresh = 3 # constante multiplicative evaluation de moy >
multiplier_tresh.threshold_value ou pas
self.multiplier_tresh_accel = 1 # constante multiplicative pour threshold acceleration
self.max_time_cut = 10 # temps max d'une coupure
self.min_time_cut = 0.1 # temps min d'au moins 4 coupures parmi toutes
self.saving = False #booléen pour savoir si on enregistre les timings IS et wake en .txt à
la fin
self.path = "/media/data-103/Nathan/Avalanche_project/aval_data/" # chemin de path
enregistrement des timings IS et wake

```

pass

```

def handle_ttl_event(self, source_node, channel, sample_number, line, state):
    """
    Handle each incoming ttl event.

    Parameters:
    source_node (int): id of the processor this event was generated from
    channel (str): name of the event channel
    sample_number (int): sample number of the event
    line (int): the line on which event was generated (0-255)
    state (bool): event state True (ON) or False (OFF)
    """
    """# ici de 0 à x alors que sur open-ephys : de 1 à x+1
    if line == 24 and state == True: #marche pour 3,4,5,6,7,8 ... un peu près tous sauf 31
par (donc bien tester)
        self.processor.add_python_event(7, True) # marche pas pour 0,1,2 | marche pour
3,4,5,6,7
    if line == 24 and state == False:
        self.processor.add_python_event(7, False)"""

    ## CREATION D'UN TABLEAU DE CONTRÔLE avec TTL TOGGLE PANEL :
    # possibilité de lancer quand on le souhaite l'intégration des threshold :
    if line == self.ligne_threshold_integration and state == True:
        self.user_thresh_go = True
        self.processor.add_python_event(3, True)

    if line == self.ligne_redo_tresh and state == True:
        self.user_redo_tresh = True
        #self.processor.add_python_event(3, True)

```

```

if line == self.ligne_stop_IS_now and state == True:
    self.user_stop_IS_now = True
    #self.processor.add_python_event(3, True)

return

def process(self, data):
    """
    Process each incoming data buffer.

    Parameters:s
    data - N x M numpy array, where N = num_channles, M = num of samples in the buffer.
    """

    ## ETAPE CONTROLE USER :

    if not self.user_thresh_go :
        return

    if self.user_redo_tresh :
        #reinitialisation du treshold et du time_counter et des listes et etats
        # probleme de comment refaire cette opération

    ## ETAPE 0 : recupere datas + calcul du time_counter + la moyenne instantanee et
    glissante + moyenne acceleration instant et glissante:

    # récupération de la channel du mean_spike_rate
    mean_spike = data[32, :] # a été fixé à la 33eme channel (MODIFIABLE)
    # recuperation des channels de l'accelerometre, place juste apres le mean firing rate (!\
    risque de chgt en fonction du xml)
    accel1 = np.mean(data[33, :])
    accel2 = np.mean(data[34, :])
    accel3 = np.mean(data[35, :])
    # calcul du temps :
    buffer_duration = mean_spike.shape[0] / self.sample_rate # duree de ce buffer
    self.time_counter += buffer_duration # on l'ajoute à notre counter

    ## calcul (maj) de la moyenne glissante :
    # moyenne du buffer courant : peut être à faire avec le min mais moins robuste
    current_mean = np.mean(mean_spike)
    # a present calcul de la moyene glissante :
    self.mean_buffers.append((buffer_duration, current_mean))
    self.mean_buffer_time += buffer_duration
    # on enleve les valeurs trop anciennes et ajout des nouvelles pour qu'on reste sur un
    temps de time_value_moy maximal

```

```

while self.mean_buffer_time > self.time_value_moy:
    dt, val = self.mean_buffers.popleft()
    self.mean_buffer_time -= dt
# on moyennise :
moy_glissante = np.mean([v for _, v in self.mean_buffers])

##calcul (maj) de la moy acceleration glissante :
#moy instant acceleration :
norm_acceleration = ((accel1**2 + accel2**2 + accel3**2)**(0.5))*10000 # on multiplie
par 10000 pour plus de lisibilite
# a present calcul de la moyenne glissante acceleration:
self.acceleration_buffers.append((buffer_duration, norm_acceleration))
self.acceleration_buffer_time += buffer_duration
# on enleve les valeurs trop anciennes et ajout des nouvelles pour qu'on reste sur un
temps de time_value_moy maximal
while self.acceleration_buffer_time > self.time_value_acceleration:
    dt, val = self.acceleration_buffers.popleft()
    self.acceleration_buffer_time -= dt
# on moyennise :
acceleration_glissante = np.mean([v for _, v in self.acceleration_buffers])

#On ajoute les valeurs du buffer tant qu'on est en dessous de la limite
integration_total_time
# ETAPE 1 : CALCUL DES TRESHOLDS
if not self.threshold_go:
    #accumulation des valeurs du buffer de mean_firing_rate :
    self.buffer_points.append(mean_spike)
    # accumulation des valeurs de l'acceleration dans ce vecteur
    self.accel_points.append(norm_acceleration)

    # si fin de la periode d'integration (x secondes)
    if self.time_counter >= self.time_limit_integration:
        self.threshold_go = True #on a calcule les treshholds

    # METHODE : On prend la valeur du xème percentile
    all_values = np.concatenate(self.buffer_points) # on concatenate la liste de listes
de valeurs
    self.threshold_value = np.percentile(all_values, self.percentage) # calcul du
pourcentile qui sera notre threshold

    # application de la eme methode pour le treshold d'acceleration:
    # pas besoin de concatener car deja dans la bonne forme
    self.threshold_acceleration = np.percentile(self.accel_points,
self.percentage_accel) # calcul du pourcentile qui sera notre threshold
    # on recupère la valeur moyenne au repos de l'acceleration (le décalage):
    self.mean_acceleration_decay = np.mean(self.accel_points)

```

```

    return #on n'engage pas la suite de la detection pour l'instant

# ETAPE 2 : CALCUL DES AVALANCHES ET DE L'INFRASLOW :

## detection du WAKE ou REM et impose reinitialisation ou non de la detection
# detection du wake :
    condition_wake = np.abs(acceleration_glissante - self.mean_acceleration_decay) >
np.abs(self.threshold_acceleration * self.multiplier_tresh_accel -
self.mean_acceleration_decay)

#detection du rem ou wake (sur la moyenne) (on observe de gros pics activité pour le
REM)
    condition_rem_wake = (moy_glissante > self.multiplier_tresh * self.threshold_value)

if condition_wake or condition_rem_wake:
    #creation de l'event :
    self.processor.add_python_event(self.ttl_wake, True)
    #reinitialisation de IS et avalanche
    self.L_time_cuts = []
    self.start_time_aval = -1
    self.start_time_cut = -1
    self.avalanche_state = False
    # enregistrement timing IS et du wake_REM
    if self.IS :
        self.L_time_IS.append(self.time_counter)
    if not self.wake_REM :
        self.L_time_wake_REM.append(self.time_counter)
    # changement en false IS :
    self.IS = False
    self.wake_REM = True
    self.processor.add_python_event(self.ttl_IS, False)
    return
else:
    # enregistrement de la donne de fin du wake_REM :
    if self.wake_REM :
        self.L_time_wake_REM.append(self.time_counter)
    # on indique qu'on est ni dans le REM ni dans le wake :
    self.wake_REM = False
    self.processor.add_python_event(self.ttl_wake, False)

# ca vaut le coup de faire l'analyse des avalanches :

## condition de STOP sur les cuts (coupures entre avalanches) trop grands:
cond_max_cut = (self.time_counter - self.start_time_cut < self.max_time_cut)
if self.IS and self.start_time_cut>=0 and not cond_max_cut:
    self.IS = False
    L_time_cuts = []

```

#on reinitialise aussi le cut pour pas qu'il soit pris en compte dans la prochaine serie de cuts entre avalanche

```
self.start_time_cut = -1
```

```
self.processor.add_python_event(self.ttl_IS, False)
```

décision avalanche

```
avalanche_now = (current_mean-self.threshold_value) > 0
```

transition OFF -> ON (presence avalanche)

```
if avalanche_now and not self.avalanche_state:
```

```
    self.avalanche_state = True
```

```
    print("AVALANCHE ON | mean =", current_mean)
```

#calcul du temps de début de l'avalanche :

```
self.start_time_aval = self.time_counter
```

#calcul du dernier cut (espace entre deux avalanches) et ajout dans L_time_cuts :

```
if self.start_time_cut >= 0:
```

```
    self.end_time_cut = self.time_counter
```

```
    time_cut = self.end_time_cut - self.start_time_cut
```

```
    self.L_time_cuts.append(time_cut)
```

#réinitialisation du début de cut :

```
    self.start_time_cut = -1
```

affichage de l'avalanche :

```
self.processor.add_python_event(self.ttl_aval, True)
```

transition ON -> OFF (fin avalanche)

```
elif not avalanche_now and self.avalanche_state:
```

```
    self.avalanche_state = False
```

```
    print("AVALANCHE OFF | mean =", current_mean)
```

on mesure le temps de l'avalanche

```
if self.start_time_aval >= 0:
```

```
    self.end_time_aval = self.time_counter
```

```
    time_aval = self.end_time_aval - self.start_time_aval
```

si c'est trop grand en temps on n'est plus dans sleep donc on arrete tout

```
if time_aval > self.max_time_aval and self.IS:
```

```
    self.L_time_cuts = []
```

```
    self.IS = False
```

#enregistrement de la fin du IS

```
    self.L_time_IS.append(self.time_counter)
```

```
    self.processor.add_python_event(self.ttl_IS, False)
```

#si on est pas dans IS mais que j'ai un trop grand on réinitialise tout

```

if time_aval > self.max_time_aval and not self.IS:

    self.L_time_cuts = []
    self.IS = False
    self.processor.add_python_event(self.ttl_IS, False)

    # sinon on continue en commençant a mesurer la coupure (cut)
    else :
        self.start_time_cut = self.time_counter

    # affichage fin de l'avalanche :
    self.processor.add_python_event(self.ttl_aval, False)


## detection InfraSlow Rythm (IS) :
if len(self.L_time_cuts) >= 4 :

    # condition 1 : aucune coupure trop longue
    cond_max = all(t < self.max_time_cut for t in self.L_time_cuts)

    # condition 2 : au moins 4 coupures suffisamment longues
    long_cuts = [t for t in self.L_time_cuts if t > self.min_time_cut]
    cond_min = len(long_cuts) >= 4

    if cond_max and cond_min:
        #print("max liste cuts: ",np.max(self.L_time_cuts))
        #enregistrement du changement :
        if not self.IS:
            self.L_time_IS.append(self.time_counter)
            self.IS = True
            self.processor.add_python_event(self.ttl_IS, True)

return

# a la fin acquisition : on enregistre les intervalles de IS et de wake_REM
def stop_acquisition(self):
    if self.saving : # si paramètre est valide, on enregistre à sur le chemin self.path
        namedossier = os.path.abspath(self.path + "IS_wake_timings.txt") #
IS_wake_timings
        with open(namedossier,"a") as f :
            line_count = sum(1 for line in fileinput.input(namedossier))
            #on numerote le nb d'acquisition stop avec les lignes et en met tt dedans :
            print("STOP ACQUISITION NUMBER",int(line_count/5) + 1,file=f)
            print("liste des timings balisant les IS (commence par un start ):",file=f)

```

```

        print(self.L_time_IS,file=f)
        print("liste des timings balisant les wake/REM intervals (commence par un start
):",file=f)
        print(self.L_time_wake_REM,file=f)

```

```

    return

```

```

    import numpy as np
    import math
    import oe_pyprocessor
    from collections import deque # utile pour la moyenne glissante
    import os
    import fileinput
    # On importe la classe :
    class PyProcessor:

```

```

    def __init__(self, processor, num_channels, sample_rate):

```

```

        """

```

A new processor is initialized whenever the plugin settings are updated

Parameters:

processor (object): Python Processor class object used for adding events from python.

num_channels (int): number of input channels in the selected stream.

sample_rate (float): sample rate of the selected stream

```

        """

```

```

        print("Num Channels: ", num_channels, " | Sample Rate: ", sample_rate)

```

On récupère les informations données par Open-ephys

```

        self.processor = processor

```

```

        self.num_channels = num_channels

```

```

        self.sample_rate = sample_rate

```

General et calcul Etape 1 calcul thresholds:

```

        self.time_counter = 0.0

```

```

        self.threshold_time_counter = 0.0

```

```

        self.buffer_points = []

```

```

        self.accel_points = []

```

```

        self.threshold_value = None

```

```

        self.threshold_acceleration = None

```

```

        self.mean_acceleration_decay = None

```

pour la moyenne glissante:

```

        self.mean_buffers = deque() # stocke (time, value)

```

```

        self.mean_buffer_time = 0

```

pour l'accélération glissante:

```

        self.acceleration_buffers = deque() #stocke (time,value)

```

```

        self.acceleration_buffer_time = 0

```

```

# boolens/etats :
self.threshold_calculated = True #mettre True    # etat on a fini le calcul des treshold
sur les x secondes nécessaires
self.avalanche_state = False # etat ON ou OFF avalanche (OFF au départ)
self.IS_state = False        # etat ON ou OFF InfraSlow Rhythm (OFF au départ)
self.wake_REM_state = False  # etat ON ou OFF Wake ou REM (OFF au départ)

#les events de python ttl : voir ETAPE 3
self.ttl_aval = 7           # ligne TTL utilisée pour etat avalanche
self.ttl_IS = 4             # ligne TTL utilisée pour etat InfraSlow
self.ttl_wake_REM = 1       # ligne TTL utilisée pour etat Wake

# la communication avec utilisateur (ttl toggle panel), /\ numero a cliquer sur TTPanel :
valeur + 1 :
self.ligne_threshold_integration = 9    # pour engager le calcul du threshold
self.ligne_stop_IS_now = 14            # pour arreter stimulation et remettre à 0
IS/avalanche states
self.user_redo_tresh = False           # etat associe a la ligne
ligne_threshold_integration
self.user_counter_decision = 0         # nombre de fois l'user a decider d'engager la
decision
self.user_stop_IS_now = False          # etat associe a la ligne ligne_stop_IS_now

# les temps d'avallanches (aval) et de coupures (cuts) etre avalanche
self.start_time_aval = -1
self.end_time_aval = 0
self.start_time_cut = -1
self.end_time_cut = 0

# listes qui enregistrent les coupures, les IS timings et les wake_REM timings :
self.L_time_cuts = []
self.L_time_IS = []
self.L_time_wake_REM = []

# parameters of limitations : possible de jouer sur ces parametres
self.time_limit_integration = 20.0 # fixé à 20 secondes par défaut pour l'instant
self.percentage = 70 # fixé à 70eme pourcentile pour percentile du treshold des valeurs
mean firing rate
self.percentage_accel = 90 # xièem pourcentile des valeurs du vecteur des valeurs de
la norm de l'acceleration
self.max_time_aval = 12 # temps d'avalanche max
self.time_value_moy = 5 # temps du calcul de la moyenne glissante (sa taille de
fenêtre)
self.time_value_acceleration = 5 # temps du calcul de la moyenne glissante de
l'acceleration (sa taille de fenêtre)

```

```

        self.multiplier_tresh = 3 # constante multiplicative evaluation de moy >
multiplier_tresh.threshold_value ou pas
        self.multiplier_tresh_accel = 1 # constante multiplicative pour threshold acceleration
        self.max_time_cut = 10 # temps max d'une coupure
        self.min_time_cut = 0.1 # temps min d'au moins 4 coupures parmi toutes
        self.saving = False #boolean pour savoir si on enregistre les timings IS et wake en .txt à
la fin
        self.path = "/media/data-103/Nathan/Avalanche_project/aval_data/" # chemin de path
enregistrement des timings IS et wake

```

pass

```

def handle_ttl_event(self, source_node, channel, sample_number, line, state):
    """
    Handle each incoming ttl event.

    Parameters:
    source_node (int): id of the processor this event was generated from
    channel (str): name of the event channel
    sample_number (int): sample number of the event
    line (int): the line on which event was generated (0-255)
    state (bool): event state True (ON) or False (OFF)
    """

    """# ici de 0 à x alors que sur open-ephys : de 1 à x+1
    if line == 24 and state == True: #marche pour 3,4,5,6,7,8 ... un peu près tous sauf 31
par (donc bien tester)
        self.processor.add_python_event(7, True) # marche pas pour 0,1,2 | marche pour
3,4,5,6,7
    if line == 24 and state == False:
        self.processor.add_python_event(7, False)"""

    ## CREATION D'UN TABLEAU DE CONTRÔLE avec TTL TOGGLE PANEL :
    # possibilite de lancer quand on le souhaite l'integration des threshold :
    if (line == self.ligne_threshold_integration and state == True) and not
self.user_redo_tresh:
        self.user_redo_tresh = True    # calcul du threshold demande
        self.user_counter_decision += 1 # +1 decision prise par le user sur le treshhold calcul
        self.processor.add_python_event(3, True)

    if (line == self.ligne_threshold_integration and state == False):
        self.processor.add_python_event(3, False)

    if line == self.ligne_stop_IS_now and state == True:
        self.user_stop_IS_now = True
        #self.processor.add_python_event(3, True)

```

```

if line == self.ligne_stop_IS_now and state == False:
    self.user_stop_IS_now = False
return

def process(self, data):
    """
    Process each incoming data buffer.

    Parameters:s
    data - N x M numpy array, where N = num_channles, M = num of samples in the buffer.
    """

    # ---- ETAPE 0 : recupere datas + calcul du time_counter + la moyenne instantanee et
    glissante + moyenne acceleration instant et glissante: ----

    # récupération de la channel du mean_spike_rate
    mean_spike = data[32, :] # a été fixé à la 33eme channel (MODIFIABLE)
    # recuperation des channels de l'accelerometre, place juste apres le mean firing rate (!\
    risque de chgt en fonction du xml)
    accel1 = np.mean(data[33, :])
    accel2 = np.mean(data[34, :])
    accel3 = np.mean(data[35, :])
    # calcul du temps :
    buffer_duration = mean_spike.shape[0] / self.sample_rate # duree de ce buffer
    self.time_counter += buffer_duration # on l'ajoute à notre counter general
    self.threshold_time_counter += buffer_duration # on incremente le timer du temps de
    calcul de threshold

    ## calcul (maj) de la moyenne glissante :
    # moyenne du buffer courant : peut être à faire avec le min mais moins robuste
    current_mean = np.mean(mean_spike)
    # a present calcul de la moyene glissante :
    self.mean_buffers.append((buffer_duration, current_mean))
    self.mean_buffer_time += buffer_duration
    # on enleve les valeurs trop anciennes et ajout des nouvelles pour qu'on reste sur un
    temps de time_value_moy maximal
    while self.mean_buffer_time > self.time_value_moy:
        dt, val = self.mean_buffers.popleft()
        self.mean_buffer_time -= dt
    # on moyennise :
    moy_glissante = np.mean([v for _, v in self.mean_buffers])

    ##calcul (maj) de la moy acceleration glisaante :
    #moy instant acceleration :
    norm_acceleration = ((accel1**2 + accel2**2 + accel3**2)**(0.5))*10000 # on multiplie
    par 10000 pour plus de lisbilite

```

```

# a present calcul de la moyenne glissante acceleration:
self.acceleration_buffers.append((buffer_duration, norm_acceleration))
self.acceleration_buffer_time += buffer_duration
# on enleve les valeurs trop anciennes et ajout des nouvelles pour qu'on reste sur un
temps de time_value_moy maximal
while self.acceleration_buffer_time > self.time_value_acceleration:
    dt, val = self.acceleration_buffers.popleft()
    self.acceleration_buffer_time -= dt
# on moyennise :
acceleration_glissante = np.mean([v for _, v in self.acceleration_buffers])

# ---- ETAPE 1: CONTROLE USER : ----

# /\ AVERTISSEMENT : si le USER clique pour engager le calcul du treshhold il a 20
sec pour décliquer le bouton sinon ca refait le calcul
# Tant que le user n'a rien demande on engage pas de calcul du python processor
if self.user_counter_decision < 1 :
    return
# /\ AVERTISSEMENT : si le USER clique pour engager le calcul du treshhold il a 20
sec pour décliquer le bouton sinon ca refait le calcul (self.user_redo_tresh)
# Si le user demande de refaire ou faire le calcul du treshhold et que c'etait deja calcule
ou premiere iteration (self.threshold_calculated)
if self.user_redo_tresh and self.user_counter_decision > 0 and
self.threshold_calculated:
    # reinitialisation des parametres pour refaire l'integration :
    self.threshold_calculated = False
    self.threshold_time_counter = 0.0
    self.buffer_points = []
    self.accel_points = []
    # reinitialisation des autres parametres de la suite (la detection de l'ETAPE 4) :
    self.avalanche_state = False
    self.IS_state = False
    self.wake_REM_state = False
    self.start_time_aval = -1
    self.end_time_aval = 0
    self.start_time_cut = -1
    self.end_time_cut = 0
    self.L_time_cuts = []

# si le user demande d'arreter l'IS :
if self.user_stop_IS_now :
    #réinitialisation des parametres de detection :
    self.avalanche_state = False
    self.IS_state = False
    self.wake_REM_state = False
    self.start_time_aval = -1
    self.end_time_aval = 0

```

```

self.start_time_cut = -1
self.end_time_cut = 0
self.L_time_cuts = []

# ---- ETAPE 2 : apparition graphique des states : ----
# mauvaise idee a ne pas faire
# apparition d'une avalanche :
self.processor.add_python_event(self.ttl_aval, self.avalanche_state)
# apparition ou non d'IS
self.processor.add_python_event(self.ttl_IS, self.IS_state)
# apparition ou non de wake ou REM
self.processor.add_python_event(self.ttl_wake_REM, self.wake_REM_state)

# ---- ETAPE 3 : CALCUL DES TRESHOLDS ----

# On ajoute les valeurs du buffer tant qu'on est en dessous de la limite
integration_total_time
if not self.threshold_calculated:
    # accumulation des valeurs du buffer de mean_firing_rate :
    self.buffer_points.append(mean_spike)
    # accumulation des valeurs de l'acceleration dans ce vecteur
    self.accel_points.append(norm_acceleration)

    # si fin de la periode d'integration (x secondes)
    if self.threshold_time_counter >= self.time_limit_integration:
        self.threshold_calculated = True # on a calcule les tresholds donc on arrete

    # METHODE : On prend la valeur du xème percentile
    all_values = np.concatenate(self.buffer_points) # on concatenate la liste de listes
    de valeurs
    self.threshold_value = np.percentile(all_values, self.percentage) # calcul du
    pourcentage qui sera notre threshold

    # application de la meme methode pour le treshold d'acceleration:
    # pas besoin de concatener car deja dans la bonne forme
    self.threshold_acceleration = np.percentile(self.accel_points,
    self.percentage_accel) # calcul du pourcentage qui sera notre threshold
    # on recupère la valeur moyenne au repos de l'acceleration (le décalage):
    self.mean_acceleration_decay = np.mean(self.accel_points)

    # on termine par reinitialiser le bouton du user :
    self.user_redo_tresh = False
    return # on n'engage pas la suite de la detection pour l'instant

# ---- ETAPE 4 : CALCUL DES AVALANCHES ET DE L'INFRASLOW : ----

```

```

## detection du WAKE ou REM et impose reinitialisation ou non de la detection
# detection du wake :
condition_wake = np.abs(acceleration_glissante - self.mean_acceleration_decay) >
np.abs(self.threshold_acceleration * self.multiplier_tresh_accel -
self.mean_acceleration_decay)

#detection du rem ou wake (sur la moyenne) (on observe de gros pics activité pour le
REM)
condition_rem_wake = (moy_glissante > self.multiplier_tresh * self.threshold_value)

#si on detecte du mouvement :
if condition_wake or condition_rem_wake:

    #reinitialisation de IS et avalanche
    self.L_time_cuts = []
    self.start_time_aval = -1
    self.start_time_cut = -1
    self.avalanche_state = False
    # enregistrement timing IS et du wake_REM
    if self.IS_state:
        self.L_time_IS.append(self.time_counter)
    if not self.wake_REM_state :
        self.L_time_wake_REM.append(self.time_counter)
        #creation de l'event ttl:
        self.processor.add_python_event(self.ttl_wake_REM, True)
    # changement en false IS :
    self.IS_state= False
    self.wake_REM_state = True
    self.processor.add_python_event(self.ttl_IS, False)
    return
# si on detecte pas de mouvement / activite hors du commun du sleep :
else:
    # enregistrement de la donnee de fin du wake_REM :
    if self.wake_REM_state :
        self.L_time_wake_REM.append(self.time_counter)
        #creation de l'event ttl :
        self.processor.add_python_event(self.ttl_wake_REM, False)
    # on indique qu'on est ni dans le REM ni dans le wake :
    self.wake_REM_state = False

# ca vaut le coup de faire l'analyse des avalanches :

## condition de STOP sur les cuts (coupures entre avalanches) trop grands:
cond_max_cut = (self.time_counter - self.start_time_cut < self.max_time_cut)
if self.IS_state and self.start_time_cut>=0 and not cond_max_cut and self.IS_state:
    self.IS_state= False

```

```
self.L_time_IS.append(self.time_counter)
self.L_time_cuts = []
#on reinitialise aussi le cut pour pas qu'il soit pris en compte dans la prochaine
serie de cuts entre avalanche
```

```
self.start_time_cut = -1
self.processor.add_python_event(self.ttl_IS, False)
```

```
# décision avalanche
avalanche_now = (current_mean-self.threshold_value) > 0
```

```
## transition OFF -> ON (presence avalanche)
if avalanche_now and not self.avalanche_state:
    self.avalanche_state = True
    print("AVALANCHE ON | mean =", current_mean)
```

```
#calcul du temps de début de l'avalanche :
self.start_time_aval = self.time_counter
#calcul du dernier cut (espace entre deux avalanches) et ajout dans L_time_cuts :
if self.start_time_cut >= 0:
```

```
    self.end_time_cut = self.time_counter
    time_cut = self.end_time_cut - self.start_time_cut
    self.L_time_cuts.append(time_cut)
    #réinitialisation du début de cut :
    self.start_time_cut = -1
```

```
# affichage de l'avalanche :
self.processor.add_python_event(self.ttl_aval, True)
```

```
## transition ON -> OFF (fin avalanche)
elif not avalanche_now and self.avalanche_state:
    self.avalanche_state = False
    print("AVALANCHE OFF | mean =", current_mean)
```

```
# on mesure le temps de l'avalanche
if self.start_time_aval >= 0:
    self.end_time_aval = self.time_counter
    time_aval = self.end_time_aval - self.start_time_aval
    # si c'est trop grand en temps on n'est plus dans sleep donc on arrete tout
    if time_aval > self.max_time_aval and self.IS_state:
```

```
        self.L_time_cuts = []
        self.IS_state= False
        #enregistrement de la fin du IS
        self.L_time_IS.append(self.time_counter)
```

```

        self.processor.add_python_event(self.ttl_IS, False)
        #si on est pas dans IS mais que j'ai un trop grand on réinitialise tout
        if time_aval > self.max_time_aval and not self.IS_state:

            self.L_time_cuts = []
            self.IS_state= False
            self.processor.add_python_event(self.ttl_IS, False)

        # sinon on continue en commençant a mesurer la coupure (cut)
        else :
            self.start_time_cut = self.time_counter

        # affichage fin de l'avalanche :
        self.processor.add_python_event(self.ttl_aval, False)

    ## detection InfraSlow Rythm (IS) :
    # si on est pas le wake_REM et qu'on a minimum 4 cuts (off)
    if len(self.L_time_cuts) >= 4 and not self.wake_REM:

        # condition 1 : aucune coupure trop longue
        cond_max = all(t < self.max_time_cut for t in self.L_time_cuts)

        # condition 2 : au moins 4 coupures suffisamment longues
        long_cuts = [t for t in self.L_time_cuts if t > self.min_time_cut]
        cond_min = len(long_cuts) >= 4

        # ca verifie les 2 conditions : alors on rentre dans l'IS
        if cond_max and cond_min:
            #print("max liste cuts: ",np.max(self.L_time_cuts))
            #enregistrement du changement :
            if not self.IS_state:
                self.L_time_IS.append(self.time_counter)
                self.processor.add_python_event(self.ttl_IS, True)
            self.IS_state= True

    return

# a la fin acquisition : on enregistre les intervalles de IS et de wake_REM
def stop_acquisition(self):
    if self.saving : # si paramètre est valide, on enregistre à sur le chemin self.path
        namedossier = os.path.abspath(self.path + "IS_wake_timings.txt") #
        IS_wake_timings
        with open(namedossier,"a") as f :
            line_count = sum(1 for line in fileinput.input(namedossier))

```

```

        #on numerote le nb d'acquisition stop avec les lignes et en met tt dedans :
        print("STOP ACQUISITION NUMBER",int(line_count/5) + 1,file=f)
        print("liste des timings balisants les IS (commence par un start ):",file=f)
        print(self.L_time_IS,file=f)
        print("liste des timings balisants les wake/REM intervals (commence par un start
):",file=f)
        print(self.L_time_wake_REM,file=f)

    return

```

CODE TEENSY :

```

#include "Cyclops.h"

Cyclops cyclops0(CH0);

// état interne
volatile bool stim_on = false;
#define FREQ 5.0f
#define AMPLITUDE 4095.0
#define PI_CONST 3.14159265358979323846

void setup()
{
    Cyclops::begin();
    cyclops0.dac_load_voltage(0);

    // déclenché à chaque changement d'état du TRIG (venant du Arduino)
    cyclops0.set_trigger(update_state, CHANGE);
}

void loop()
{
    if (stim_on) // Si on détecte le trigger
    {
        // calcul du temps
        float time = millis() / 1000.0f;
        // calcul du sinus associé (sin(2pift))
        float sine = sinf(FREQ*2*PI_CONST*time);
        // on le normalise pour que le sinus soit au dessus de 0 (pas de
        négatif)
        float normalized = 0.5f*(sine + 1.0f);
        // on calcule le voltage avec amplitude
        unsigned int voltage = (unsigned int)(AMPLITUDE * (normalized));
        // on envoie cette tension
    }
}

```

```
        cyclops0.dac_load_voltage(voltage);
    }
    else
    {    // pas de trigger : pas de stimulation :
        cyclops0.dac_load_voltage(0);
    }
}

void update_state()
{
    // On inverse l'état à chaque changement
    stim_on = !stim_on;
}
```